

```
# ===== main.py (926 行) =====
# =====
# 医学影像工作站 Pro + 重建实验室
# Medical Imaging Workstation Pro + Recon Lab
#
# 技术栈: PySide6 (Qt6 Python绑定) + NumPy + pydicom + scikit-image
# 架构: 多文件模块化
#   ai_engine.py      - AutoAIEngineThread (后台 AI 推理线程)
#   graphics_view.py - MedicalGraphicsView (影像交互视图组件)
#   recon.py         - 纯计算重建算法 (无 Qt 依赖)
#   main.py          - MedicalViewer 主窗口 + 入口
# =====

import json
import os
import re
import sys
from concurrent.futures import ThreadPoolExecutor

import numpy as np
import pydicom # 读取 DICOM 医学影像文件格式
from PySide6.QtCore import Qt, QTimer
from PySide6.QtGui import QImage, QPixmap
from PySide6.QtWidgets import QApplication, QFileDialog, QMainWindow, QMessageBox

# 子模块导入
import mpr_geometry
from ai_engine import AutoAIEngineThread
from annotation_lab import AnnotationMixin
from compare_lab import CompareMixin
from constants import (
    AXIAL,
    CORONAL,
    LABEL_LUT,
    LABELS_JSON,
    MANUAL_TRACK_LABEL,
    SAGITTAL,
    TOOL_POINTER,
)
from interaction import InteractionMixin
from recon_lab import ReconLabMixin
from ui_builder import UiBuilderMixin

# AutoAIEngineThread -> 已移至 ai_engine.py
# =====
# 主窗口: 医学影像工作站
# 负责: DICOM 加载、多平面临床阅片渲染、窗位/工具/布局/AI 调度、i18n 重译、键盘导航。
# UI 构建见 UiBuilderMixin (ui_builder.py), 重建实验室见 ReconLabMixin
# (recon_lab.py), 双序列对比见 CompareMixin (compare_lab.py), 标注/分割/
```

```

# 器官定量见 AnnotationMixin (annotation_lab.py), Cine/MPR 交互见
# InteractionMixin (interaction.py)。KeyPressEvent 因 Qt 重写需留本体 (MRO)。
# =====
class MedicalViewer(QMainWindow, ReconLabMixin, CompareMixin, AnnotationMixin,
                    UiBuilderMixin, InteractionMixin):
    # 临床标准窗宽/窗位预设值 (中英文键名均支持, 兼容语言切换后的下拉选项)
    # 提为类级常量避免在每次 update_display 中重新构造 (MPR 多窗模式下每帧 4 次浪费)
    _WW_PRESETS = {"Lung": 1500, "Medi": 400, "Bone": 1500, "Vasc": 600, "Abdo": 150, "Brain": 80,
                  "肺窗": 1500, "纵隔": 400, "骨窗": 1500, "血管": 600, "腹部": 150, "脑窗": 80}
    _WL_PRESETS = {"Lung": -500, "Medi": 40, "Bone": 400, "Vasc": 150, "Abdo": 30, "Brain": 40,
                  "肺窗": -500, "纵隔": 40, "骨窗": 400, "血管": 150, "腹部": 30, "脑窗": 40}

    def __init__(self, data_dir=None):
        super().__init__()
        self.setWindowTitle("Medical Imaging Workstation Pro + Recon Lab")
        self.resize(1600, 950)

        # --- 影像数据 ---
        self.dicom_datasets = []          # 按 Z 轴位置排序的 pydicom Dataset 列表
        self.current_slice_idx = 0        # 当前显示的切片索引 (冗余字段, 实际以 current_3d_pos[0] 为准)
        self.views = {}                   # {vid: {'container', 'view', 'cb_plane', ...}} 视图字典

        # --- 工具与标注 ---
        self.active_tool = TOOL_POINTER
        # global_annotations 结构: {'all': [全局标注列表], slice_idx: [该切片标注列表]}
        # 'all' 键下的标注会穿透所有切片显示 (由 chk_global_scope 控制新标注归属)
        self.global_annotations = {'all': []}

        # --- 3D 体数据 ---
        self.volume_hu = None              # 完整 HU 值体素数组 shape=(Z, H, W), float32
        self.volume_mask = None            # AI 多器官标签图, shape=(Z,H,W) uint8: 0=背景,1-24=器官,255=手动追踪
        self.organ_names = self._load_organ_labels() # {类别号: (中文名, 英文名)}, 用于图例
        self._organ_stats = []             # 最近一次器官定量结果, 供面板显示与 CSV 导出
        self._hidden_organ = set()         # 被用户在图例中点隐的器官类别, 渲染时跳过
        self._mask_undo = []              # 分割编辑撤销栈: [(切片号, 编辑前蒙版切片)], 上限 20
        self.is_english = False            # 界面语言, False=中文, True=英文
        self.anonymize = False             # 脱敏模式: 显示层隐去患者身份信息 (不改底层 DICOM)
        self.current_3d_pos = [0, 0, 0]    # [z, y, x], MPR 联动的三维光标位置

        # --- 双序列随访对比状态 (轻量版: 独立模式, 不改单体数据模型) ---
        self.compare_mode_active = False   # 是否处于对比模式, 影响 update_display 分支
        self.compare_volume = None         # 对比 (既往) 序列的 HU 体数据, shape=(Z,H,W)
        self.compare_datasets = []         # 对比序列的 pydicom Dataset 列表
        self._pre_compare_layout = 0       # 进入对比模式前的布局, 退出时还原
        self._primary_zpos = None           # 主序列各层的解剖 z 坐标(mm), 用于对比配准
        self._compare_zpos = None          # 对比序列各层的解剖 z 坐标(mm)

        # --- Cine 电影播放 ---
        self.cine_timer = QTimer(self)     # 自动连续翻片定时器
        self.cine_timer.timeout.connect(self._cine_step)

```

```

self._cine_dir = 1                # 往返(bounce)播放方向: +1 向下 / -1 向上

# --- 重建实验室状态 ---
self.recon_mode_active = False    # 是否处于重建实验室 Tab, 影响 update_display() 的行为分支
self._pre_recon_layout = 0        # 进入重建实验室前的布局模式, 退出时用于还原
self._recon_ref_z = None          # V1 参考图上次渲染的切片号; 仅当它改变才重置重建流水线
self.current_sinogram = None      # 当前切片的弦图 (Radon 变换结果), shape=(detectors, angles)
self.current_theta = None         # 弦图对应的角度数组, 单位为度
self._last_recon_img = None       # 最近一次矩阵重建结果 (n×n, 未放大), 作为下次生成弦图的输入源

# --- AI 引擎状态 ---
self.ai_thread = None             # AutoAIEngineThread 实例
self._ai_generation: 每次加载新数据自增, 回调中对比该值可丢弃旧数据的结果 (竞态保护)
self._ai_generation = 0
self._ai_state = 'standby'        # 'standby' | 'running' | 'done'
self._ai_time_ms = 0.0           # 最近一次 AI 推理耗时 (毫秒)

# --- 系统矩阵缓存 ---
# DMR/ART 都需要构建 A 矩阵, 计算代价极高 (O(n²) 次 Radon 变换)
# 当图像尺寸和角度配置不变时, 直接复用缓存, 避免重复等待
self._cached_A = None            # 缓存的系统矩阵 A, shape=(n_rays, n×n)
self._cached_A_key = None        # 缓存的 key=(n, len(theta), theta[0], theta[-1])

# BP 结果缓存: FBP 需要先运行 BP, 两者共享缓存避免重复计算
self._cached_bp = None           # 缓存的 BP 重建结果
self._cached_bp_sino = None      # 缓存对应的弦图对象引用 (用 is 比较, 避免 id() 回收复用风险)

self.setup_stylesheet()
self.init_ui()
self.update_language()

# 延迟 50ms 执行布局切换, 确保 Qt 窗口几何完成初始化再设置 splitter 尺寸
QTimer.singleShot(50, lambda: self.switch_layout(0))

# 可选: 启动时加载指定 DICOM 目录 (由入口 --data 传入或测试显式指定); 默认不加载。
# 不再硬编码自动加载本地"肺癌"目录—避免开发便利泄进产品入口, 也避免误加载患者数据。
if data_dir and os.path.isdir(data_dir):
    self.load_data(data_dir)

def _load_organ_labels(self):
    """加载器官名表 organ_labels_candidate.json; 缺失时回退到内置高置信名称。
    映射已确证=TotalSegmentator class_map_part_organ (见该 JSON 的 _meta 与 experiments/)。"""
    fallback = {5: ("肝", "Liver"),
                 10: ("左肺上叶", "Lung UL (L)"), 11: ("左肺下叶", "Lung LL (L)"),
                 12: ("右肺上叶", "Lung UL (R)"), 13: ("右肺中叶", "Lung ML (R)"),
                 14: ("右肺下叶", "Lung LL (R)"),
                 MANUAL_TRACK_LABEL: ("手动追踪", "Manual")}
    try:
        with open(LABELS_JSON, encoding='utf-8') as f:
            data = json.load(f)

```

```

        names = {int(k): (v.get("name_zh", f"类{k}"), v.get("name_en", f"cls{k}"))
                  for k, v in data.get("labels", {}).items()}
        names[MANUAL_TRACK_LABEL] = ("手动追踪", "Manual")
        return names
    except Exception:
        return fallback

def toggle_language(self):
    """切换中英文界面, 然后刷新所有控件文字。"""
    self.is_english = not self.is_english
    self.update_language()

@staticmethod
def _retranslate_combo(combo, en_items, cn_items, e, idx=None):
    """重译纯文本下拉框, 保留选中项 (blockSignals 防止触发回调)。
    idx=None 时沿用下拉框当前索引; 显式传入用于按外部状态 (如视图 plane) 恢复。"""
    if idx is None:
        idx = combo.currentIndex()
    idx = max(0, idx)
    combo.blockSignals(True); combo.clear()
    combo.addItems(en_items if e else cn_items)
    combo.setCurrentIndex(idx); combo.blockSignals(False)

def update_language(self):
    """按当前语言(self.is_english)重译所有常驻控件文字。
    绝大多数静态文案集中在下面的 (控件, 英文, 中文) 表里—增/改一条字符串只需
    在表中加一行, 避免逐行 setText 那样容易漏译 (历史上曾漏译 chk_global_scope)。
    含状态/索引逻辑的控件 (下拉框、播放/对比/AI 状态等) 在表后单列处理。"""
    e = self.is_english
    self.btn_lang.setText("中" if e else "EN")

# 静态文案表: (控件, 英文, 中文) — setText 类
for w, en, cn in (
    (self.tool_btns['btn_ptr'], "Pan\nProbe", "探针\n拖拽"),
    (self.tool_btns['btn_rul'], "Ruler\nDist", "测距\n卡尺"),
    (self.tool_btns['btn_drw'], "Draw\nPath", "自由\n画笔"),
    (self.tool_btns['btn_rec'], "Rect\nCrop", "矩形\n截取"),
    (self.tool_btns['btn_las'], "Lasso\nMask", "套索\n抠图"),
    (self.tool_btns['btn_trk'], "3D\nTrack", "3D\n追踪"),
    (self.tool_btns['btn_brush'], "Seg\nBrush", "分割\n画笔"),
    (self.tool_btns['btn_erase'], "Seg\nErase", "分割\n橡皮"),
    (self.tool_btns['btn_roi'], "ROI\nStats", "ROI\n密度"),
    (self.btn_import, "Load DICOM Folder", "加载 DICOM 目录"),
    (self.btn_save_proj, "Save Project", "保存标注工程"),
    (self.btn_gen_sino, "Generate Sinogram", "发射射线生成弦图"),
    (self.lbl_oversample, "Sampling:", "采样密度:"),
    (self.btn_dfr, "Direct Fourier (DFR)", "直接傅里叶重建 (DFR)"),
    (self.btn_bp, "Back Projection (BP)", "反投影法 (BP - 未滤波)"),
    (self.lbl_filter_text, "Filter:", "选择滤波器:"),
    (self.btn_fbp, "Filtered BP (FBP)", "滤波反投影 (FBP) 对比"),

```

```

        (self.lbl_matrix_size, "Image Size:", "图像尺寸:"),
        (self.lbl_art_method, "Method:", "迭代方法:"),
        (self.lbl_art_iter, "Iterations:", "迭代次数:"),
        (self.btn_dmr, "Direct Matrix Recon (DMR)", "直接矩阵重建 (DMR)"),
        (self.btn_art, "ART / SIRT Iterative", "ART / SIRT 迭代重建"),
        (self.lbl_brush, "Brush R:", "画笔半径:"),
        (self.lbl_paint_target, "Paint as:", "画笔目标:"),
        (self.btn_export_stats, "Export Stats CSV", "导出定量 CSV"),
        (self.lbl_disclaimer,
         "\u25b6 AI results & organ labels are auto-inferred – for reference only, not for diagnosis.",
         "\u25b6 AI 结果与器官标签为自动推断, 仅供参考, 非诊断依据。"),
        (self.btn_clear_anno, "Clear Mask", "清空蒙版与标注"),
        (self.btn_reset, "Reset Workspace", "重置工作区"),
        (self.lbl_ww_hint, "Right-drag on image to adjust WW/WL", "在图像上右键拖拽可快速调节窗宽/窗位"),
        (self.chk_overlay, "Overlay", "信息叠加"),
        (self.chk_invert, "Invert", "反色"),
        (self.chk_anon, "De-ID", "脱敏"),
        (self.chk_global_scope, "New anno -> all slices", "新标注穿透所有切片"),
    ):
        w.setText(en if e else cn)

# 分组框标题表: (分组框, 英文, 中文) — setTitle 类
for g, en, cn in (
    (self.grp_proj, "Projection Generation", "X射线投影生成"),
    (self.grp_algo, "Reconstruction Algorithms", "图像重建算法"),
    (self.grp_matrix, "Matrix Recon & ART / SIRT", "直接矩阵重建 & ART / SIRT"),
    (self.grp_mon, "Performance Monitor", "算法性能监控"),
    (self.grp_patient, "PATIENT INFO", "患者信息"),
    (self.grp_display, "DISPLAY CONTROL", "显示控制"),
    (self.grp_measure, "MEASURE & CLEAN", "测量与清理"),
    (self.grp_ai, "Automated AI Engine", "自动化 AI 引擎"),
):
    g.setTitle(en if e else cn)

# 工具按钮悬停提示 (表驱动)
_tips = {
    'btn_ptr': ("Pan & probe – drag to pan, click to measure HU, right-drag to adjust WW/WL",
               "探针/拖拽 – 拖动平移 | 点击测量HU值 | 右键拖拽调节窗宽窗位"),
    'btn_rul': ("Ruler – drag to measure distance (mm)", "测距卡尺 – 拖出直线测量两点距离(mm)"),
    'btn_drw': ("Freehand draw – annotate freely", "自由画笔 – 在图像上自由绘制标注"),
    'btn_rec': ("Rect crop – select ROI to export stats", "矩形截取 – 框选区域导出ROI统计"),
    'btn_las': ("Lasso mask – polygon segmentation", "套索抠图 – 绘制多边形生成分割蒙版"),
    'btn_trk': ("3D track – track structure through slices", "3D追踪 – 框选区域执行三维连通域追踪"),
    'btn_brush': ("Seg brush – paint on the current Axial slice to add to the mask",
                 "分割画笔 – 在当前横断面涂画, 补入分割蒙版 (修正 AI 遗漏)"),
    'btn_erase': ("Seg erase – wipe mask (incl. AI errors) under the stroke",
                 "分割橡皮 – 擦除涂过处的蒙版 (可清除 AI 误分割)"),
    'btn_roi': ("ROI density – drag an ellipse to read mean/SD/min/max HU & area",
               "ROI 密度 – 拖出椭圆读取内部 均值/标准差/最值 HU 及面积"),
}

```

```

for key, (tip_en, tip_cn) in _tips.items():
    self.tool_btns[key].setToolTipTip(tip_en if e else tip_cn)

self.tabs.setTabText(0, "Clinical Mode" if e else "临床阅片")
self.tabs.setTabText(1, "Recon Lab" if e else "重建实验室")

# 保留选中索引的纯文本下拉框
self._retranslate_combo(self.combo_oversample,
    ["Std 1x", "High 2x", "Ultra 4x"], ["标准 1x", "高 2x", "超高 4x"], e)
self._retranslate_combo(self.combo_layout,
    ["1x1 Single", "1x2 Dual", "2x2 Grid"],
    ["单窗模式 (1x1)", "双窗对比 (1x2)", "四窗矩阵 (2x2)"], e)

# 运行耗时占位 (仅占位态需重译, 已有结果不动)
if "耗时: --" in self.lbl_time.text() or "Time: --" in self.lbl_time.text():
    self.lbl_time.setText("Run Time: -- ms" if e else "运行耗时: -- ms")
self._update_organ_stats() # 语言切换后按新语言重渲染定量面板

# AI 状态文案随状态机
if self._ai_state == 'standby':
    self.lbl_ai_status.setText("Status: Standby" if e else "状态: 待机中")
elif self._ai_state == 'running':
    self.lbl_ai_status.setText("Processing AI Pipeline..." if e else "状态: AI 引擎自动运算中...")
elif self._ai_state == 'done':
    self.lbl_ai_status.setText(self._ai_done_text())

# 状态相关按钮
mpr_on = self.btn_mpr.isChecked()
self.btn_mpr.setText(("MPR Link: ON" if mpr_on else "MPR Link: OFF") if e
    else ("MPR 联动: 开启" if mpr_on else "MPR 联动: 关"))
for b, n in zip(self.preset_btns,
    (["Lung", "Medi", "Bone", "Vasc", "Abdo", "Brain"] if e
    else ["肺窗", "纵隔", "骨窗", "血管", "腹部", "脑窗"]), strict=False):
    b.setText(n)
self.btn_compare.setText(("Exit Compare" if self.compare_mode_active else "Load Comparison") if e
    else ("退出对比" if self.compare_mode_active else "加载对比序列"))
self.btn_cine.setText(("⏸ Pause" if self.cine_timer.isActive() else "▶ Play") if e
    else ("⏸ 暂停" if self.cine_timer.isActive() else "▶ 播放"))
# Cine 速度下拉 (item 带 ms 数据, 单列处理)
cs = max(0, self.cb_cine_speed.currentIndex())
self.cb_cine_speed.blockSignals(True); self.cb_cine_speed.clear()
for _nm, _ms in ((("Slow" if e else "慢"), 250), (("Med" if e else "中"), 120), (("Fast" if e else "快"), 50)):
    self.cb_cine_speed.addItem(_nm, _ms)
self.cb_cine_speed.setCurrentIndex(cs); self.cb_cine_speed.blockSignals(False)

# 每视图的平面/窗位下拉 + 显示/锁定复选 (cb_plane 按 vdata['plane'] 恢复, 非下拉当前索引)
v_en = ["Global", "Lung", "Medi", "Bone", "Vasc", "Abdo", "Brain"]
v_cn = ["跟随", "肺窗", "纵隔", "骨窗", "血管", "腹部", "脑窗"]
plane_en = ["Axial", "Coronal", "Sagittal"]
plane_cn = ["横断面", "冠状面", "矢状面"]

```

```

    for vdata in self.views.values():
        self._retranslate_combo(vdata['cb_plane'], plane_en, plane_cn, e, idx=vdata['plane'])
        self._retranslate_combo(vdata['preset'], v_en, v_cn, e)
        vdata['chk_anno'].setText("Anno" if e else "显示")
        vdata['lock'].setText("Lock" if e else "锁定")

    self._refresh_patient_info() # 脱敏占位文字随语言刷新
    self.on_slice_changed(self.slider_slice.value())

def on_tab_changed(self, index):
    """Tab 切换回调: 在临床阅片 (index=0) 和重建实验室 (index=1) 之间切换。

    反闪烁设计: setUpdatesEnabled(False) 屏蔽所有绘制事件, 整个切换过程只产生最终
    一帧; try/finally 确保异常时 UI 也能恢复正常刷新。具体进入 / 退出逻辑分别
    委托给 _enter_recon_mode / _exit_recon_mode。
    """
    self._stop_cine() # 切 Tab 停止 Cine 播放
    # 切换到重建实验室前先退出对比模式 (对比是临床模式专属, 还原布局避免冲突)
    if index == 1 and self.compare_mode_active:
        self._exit_compare_mode()
    self.recon_mode_active = (index == 1)
    self.setUpdatesEnabled(False)
    try:
        if self.recon_mode_active:
            self._enter_recon_mode()
        else:
            self._exit_recon_mode()
    finally:
        self.setUpdatesEnabled(True)

def _apply_grid_visibility(self, mode):
    """根据布局模式 (0=单窗, 1=双窗, 2=四窗) 调整 V2/V3/V4 与 bottom_splitter 可见性。
    仅处理 show/hide, 不设置 splitter 尺寸: 由调用方按上下文决定同步 / 异步执行 setSizes,
    避免在闪烁控制 (setUpdatesEnabled) 外多调用一次 setSizes 引发额外重绘。
    """
    vs = [self.views[i]['container'] for i in range(1, 5)]
    if mode == 0:
        vs[1].hide(); vs[2].hide(); vs[3].hide(); self.bottom_splitter.hide()
    elif mode == 1:
        vs[1].show(); vs[2].hide(); vs[3].hide(); self.bottom_splitter.hide()
    else:
        vs[1].show(); vs[2].show(); vs[3].show(); self.bottom_splitter.show()

def _apply_grid_sizes(self, mode):
    """根据布局模式设置 splitter 尺寸 (1=均分上行, 2=三个 splitter 全部均分; 0=无)。"""
    if mode == 1:
        self.top_splitter.setSizes([1000, 1000])
    elif mode == 2:
        self.top_splitter.setSizes([1000, 1000])
        self.bottom_splitter.setSizes([1000, 1000])

```

```

        self.main_splitter.setSizes([1000, 1000])

def set_view_title(self, vid, title):
    """更新指定视图工具栏中的标题标签文字。
    直接使用 create_independent_view 中缓存的 label 引用，避免每次 findChild 遍历视图树。
    """
    try:
        self.views[vid]['title_label'].setText(title)
    except Exception as e:
        print(f"Warning: set_view_title V{vid}: {e}")

def on_window_changed_by_mouse(self, delta_ww, delta_wl):
    """右键拖拽调节窗宽/窗位。
    拖拽时若当前视图使用预设窗（非"跟随"），自动重置为全局跟随模式，
    防止预设窗覆盖手动调节的值（否则 update_display 会用预设覆盖 slider）。
    """
    if not self.dicom_datasets or self.recon_mode_active:
        return
    new_ww = max(self.slider_ww.minimum(), min(self.slider_ww.maximum(), self.slider_ww.value() + delta_ww))
    new_wl = max(self.slider_wl.minimum(), min(self.slider_wl.maximum(), self.slider_wl.value() + delta_wl))
    self.slider_ww.setValue(new_ww)
    self.slider_wl.setValue(new_wl)
    for vdata in self.views.values():
        if vdata['container'].isHidden():
            continue
        if vdata['preset'].currentText() not in ["Global", "跟随"]:
            # blockSignals 防止 setCurrentIndex 触发 update_display 重入
            vdata['preset'].blockSignals(True)
            vdata['preset'].setCurrentIndex(0)
            vdata['preset'].blockSignals(False)

def change_active_tool(self, tid):
    """切换全局工具，并同步更新所有视图的 current_tool，确保各视图行为一致。"""
    self.active_tool = tid
    for v in self.views.values():
        v['view'].current_tool = tid

def _set_brush_radius(self, r):
    """同步所有视图的分割修正画笔/橡皮半径。"""
    for v in self.views.values():
        v['view'].brush_radius = r

def keyPressEvent(self, event):
    """键盘翻页：↓/PageDown 下一层，↑/PageUp 上一层，空格切换 Cine，Ctrl+Z 撤销分割编辑。"""
    if event.key() == Qt.Key_Z and (event.modifiers() & Qt.ControlModifier):
        self._undo_mask_edit(); return
    if self.volume_hu is not None and not self.recon_mode_active:
        k = event.key()
        if k in (Qt.Key_Down, Qt.Key_PageDown):
            self.slider_slice.setValue(min(self.slider_slice.value() + 1, self.slider_slice.maximum())); return

```



```

        if k in (Qt.Key_Up, Qt.Key_PageUp):
            self.slider_slice.setValue(max(self.slider_slice.value() - 1, 0)); return
        if k == Qt.Key_Space:
            self.toggle_cine(); return
        super().keyPressEvent(event)

def reset_all_states(self):
    """重置工作区到初始状态：恢复单窗布局、默认窗宽窗位、清空所有标注和弦图缓存。
    注意：仅在临床阅片模式（非重建实验室）下调用 update_display，
    避免在重建实验室中意外清空正在查看的重建结果。
    """
    self._stop_cine() # 重置时停止 Cine 播放
    if self.compare_mode_active:
        self._exit_compare_mode() # 重置时退出对比模式
    # 布局复位仅在临床模式：recon 模式需保持 2x2，改 combo_layout 会隐藏 V2/3/4
    if not self.recon_mode_active:
        self.combo_layout.setCurrentIndex(0)
    self.slider_wv.setValue(1500); self.slider_wl.setValue(-500)
    self.tool_btns['btn_ptr'].setChecked(True); self.change_active_tool(0)
    self.global_annotations = {'all': []}
    if self.volume_mask is not None:
        self.volume_mask = np.zeros(self.volume_hu.shape, dtype=np.uint8)
    self._hidden_organs.clear()
    self._mask_undo = [] # 重置清撤销栈，避免撤销回被清掉的编辑
    self.lbl_hud.setText("") # 清除光标 HUD 残留文本
    self._update_organ_stats() # 蒙版已清，定量面板同步清空
    self.btn_mpr.setChecked(False)
    self.current_sinogram = None; self.current_theta = None
    self._last_recon_img = None
    for b in [self.btn_dfr, self.btn_bp, self.btn_fbp]: b.setEnabled(False)
    # DMR/ART 只要有 DICOM 数据就可以运行（不依赖弦图）
    has_data = self.volume_hu is not None
    for b in [self.btn_dmr, self.btn_art]: b.setEnabled(has_data)
    for v in self.views.values():
        v['cb_plane'].setCurrentIndex(AXIAL)
        v['preset'].setCurrentIndex(0); v['lock'].setChecked(False)
        v['chk_anno'].setChecked(True)
        v['view']._user_zoomed = False
        v['view'].fitInView(v['view'].scene.sceneRect(), Qt.KeepAspectRatio)
    if not self.recon_mode_active:
        self.update_display()

def set_window(self, ww, wl):
    """快捷设置窗宽/窗位（供预设按钮调用），触发 slider.valueChanged → update_display。"""
    self.slider_wv.setValue(ww)
    self.slider_wl.setValue(wl)

def switch_layout(self, m):
    self._apply_grid_visibility(m)
    # setSizes 和 fitInView 合并到同一帧执行，消除两步之间的闪烁间隙

```

```

def _settle():
    self._apply_grid_sizes(m)
    for vd in self.views.values():
        v = vd['view']
        px = v.image_item.pixmap()
        # 只在 pixmap 真实存在时才 fitInView, 避免对已清空的视图操作导致 m11 被改变
        if not vd['container'].isHidden() and px and not px.isNull():
            v.fitInView(v.scene.sceneRect(), Qt.KeepAspectRatio)
    QTimer.singleShot(0, _settle)

def load_data(self, path):
    """加载 DICOM 目录并构建 3D 体积—分四步: 读盘 / 构 HU / 加载注解 / 启动 AI。"""
    self._stop_cine() # 换病例停止 Cine 播放
    if self.compare_mode_active:
        self._exit_compare_mode() # 新主序列作废旧的对比配对
    # 记住加载前状态: 若新目录无法解码, 恢复原序列而非留下 dicom_datasets 与 volume_hu
    # 不一致的半更新状态 (否则后续按 idx 取切片会越界崩溃)。
    prev_datasets, prev_volume = self.dicom_datasets, self.volume_hu
    if not self._read_dicom_dir(path):
        return
    pid = self._build_volume_hu()
    if pid is None:
        self.dicom_datasets, self.volume_hu = prev_datasets, prev_volume
        QMessageBox.warning(self, "Load Failed" if self.is_english else "加载失败",
            "No decodable image slices in this series."
            if self.is_english else "该序列没有可解码的图像切片。")
        return
    self._load_annotations_json(pid)
    mask_restored = self._load_saved_mask(pid) # 在首次显示前恢复上次的分割

    z = self.volume_hu.shape[0]
    self.on_slice_changed(z // 2)
    for b in [self.btn_dmr, self.btn_art]:
        b.setEnabled(True)
    for vd in self.views.values():
        vd['view'].user_zoomed = False # 新病例回到适配状态
    # 延迟 100ms 做 fitInView, 确保 Qt 已完成首次绘制布局再计算缩放
    QTimer.singleShot(100, lambda: [
        vd['view'].fitInView(vd['view'].scene.sceneRect(), Qt.KeepAspectRatio)
        for vd in self.views.values() if not vd['container'].isHidden()
    ])
    if mask_restored:
        # 已从磁盘恢复分割, 跳过 ~100s 的 AI 重算
        self._ai_state = 'done'
        self._ai_time_ms = 0.0
        self.lbl_ai_status.setStyleSheet("color: #00FF00; font-weight: bold;")
        self.lbl_ai_status.setText(self._ai_done_text())
        self._update_organ_stats()
    else:
        self._kickoff_ai()

```

```

def _read_dicom_dir(self, path):
    """递归扫描目录并并行读取所有 DICOM 文件，按 Z 物理位置排序。

    并行策略：用线程池 dcmread 各文件—pydicom 内部 IO + 大量 numpy 解码会释放 GIL，
    线程池在 SSD 上对千张切片可获 4-8× 加速。读盘失败的单个文件静默跳过，
    最终顺序与单线程版本严格一致（统一在所有线程完成后按 Z 物理位置排序）。

    DICOM 排序策略：
        优先使用 ImagePositionPatient[2] (床位 Z 坐标，单位 mm，物理精确)；
        若缺失该 tag，回退到 InstanceNumber (序列编号，精度较低但通用)。
    """
    # 第一阶段：列出所有候选文件（跳过 macOS 隐藏文件）
    file_paths = []
    for r, _d, fs in os.walk(path):
        for f in fs:
            if not f.startswith('.'):
                file_paths.append(os.path.join(r, f))

    # 第二阶段：线程池并行 dcmread；max_workers 上限设为 16 避免过多线程导致上下文切换开销
    def _safe_read(fp):
        try:
            return pydicom.dcmread(fp)
        except Exception:
            return None

    with ThreadPoolExecutor(max_workers=min(16, (os.cpu_count() or 4) * 2)) as ex:
        results = list(ex.map(_safe_read, file_paths))

    # 过滤掉读取失败及不含像素数据的文件 (DICOMDIR / RTSTRUCT 等)
    datasets = [ds for ds in results if ds is not None and 'PixelData' in ds]
    if not datasets:
        return False

    # 多序列目录：按 SeriesInstanceUID 分组，只保留切片最多的序列，
    # 避免把不同序列（定位像、不同重建核等）混叠进同一个体积
    from collections import defaultdict
    groups = defaultdict(list)
    for ds in datasets:
        groups[str(getattr(ds, 'SeriesInstanceUID', ''))].append(ds)
    if len(groups) > 1:
        datasets = max(groups.values(), key=len)
        print(f"检测到 {len(groups)} 个序列，选用切片最多的 ({len(datasets)} 张) ")

    # 形状一致性过滤：即使同一 SeriesInstanceUID，个别切片的矩阵尺寸也可能不同
    # （扫描中途换重建矩阵）；更常见的是 SeriesInstanceUID 缺失把多个真实序列混成一组。
    # 混合形状会让后续 np.array 堆叠抛 ValueError (主路径 _build_volume_hu 崩溃、
    # 对比路径被 try/except 误判为"无法读取")。按 (Rows, Columns) 保留数量最多的尺寸，
    # 与上面"选切片最多的序列"同一取舍思路。Rows/Columns 是含 PixelData 时的必填 tag。
    shape_groups = defaultdict(list)

```

```

for ds in datasets:
    shape_groups[(int(getattr(ds, 'Rows', 0)), int(getattr(ds, 'Columns', 0)))].append(ds)
if len(shape_groups) > 1:
    datasets = max(shape_groups.values(), key=len)
    r0, c0 = getattr(datasets[0], 'Rows', '?'), getattr(datasets[0], 'Columns', '?')
    print(f"检测到 {len(shape_groups)} 种切片尺寸, 选用数量最多的 ({len(datasets)} 张 {r0}x{c0}) ")
self.dicom_datasets = datasets

# 排序键必须在整个序列内保持一致: z 物理坐标(mm)与 InstanceNumber(序号)是不同量纲,
# 逐切片回退 (部分切片缺 ImagePositionPatient) 会把缺位置信息的层按序号插进有位置
# 信息的层之间, 打乱解剖顺序。因此先做序列级判定—所有切片都含 z 坐标才按 z 排序,
# 否则整列统一回退 InstanceNumber (序列内单调的采集序号)。
def _has_ipp(ds):
    try:
        float(ds.ImagePositionPatient[2])
        return True
    except Exception:
        return False

if all(_has_ipp(ds) for ds in self.dicom_datasets):
    self.dicom_datasets.sort(key=lambda ds: float(ds.ImagePositionPatient[2]))
else:
    self.dicom_datasets.sort(key=lambda ds: int(getattr(ds, 'InstanceNumber', 0)))
return True

def _build_volume_hu(self):
    """从 dicom_datasets 构建 3D HU 数组, 初始化蒙版、3D 光标、切片滑动条。
    成功返回 PatientID; 无任何可解码切片时返回 None (由 load_data 提示并中止)。

    HU 值转换公式 (DICOM 标准):
    HU = pixel_value × RescaleSlope + RescaleIntercept
    典型值: Slope=1, Intercept=-1024 (GE 扫描仪常见), 使得空气≈-1000 HU
    """
    ds = self.dicom_datasets[0]
    pid = str(getattr(ds, 'PatientID', 'N/A'))

    # 逐片解码并转 HU, 防御式处理畸形数据 (一张坏片不带崩整卷):
    # - pixel_array 解码失败 (PixelData 截断 / 压缩语法缺编解码器 / group 0028 非法) → 跳过该片;
    # - 多帧文件 (pixel_array 为 3D, NumberOfFrames>1) → 展开为多个 2D 帧, 共用该文件元数据。
    # dicom_datasets 随之同步为实际保留/展开后的切片, 保证按 idx 取元数据与体积层一一对应。
    frames, kept = [], []
    for d in self.dicom_datasets:
        try:
            arr = d.pixel_array
        except Exception as e:
            print(f"跳过无法解码的切片: {e}")
            continue
        hu = (arr.astype(np.float32) * self._dcm_float(d, 'RescaleSlope', 1.0)
              + self._dcm_float(d, 'RescaleIntercept', 0.0))
        if hu.ndim == 2:

```

```

        frames.append(hu); kept.append(d)
    elif hu.ndim == 3:          # 多帧: 逐帧展开为切片
        for fr in hu:
            frames.append(fr); kept.append(d)
    # 其余维度 (异常数据) 忽略
if not frames:
    return None # 无任何可解码切片
# 兜底: 万一帧尺寸仍不齐 (多帧与单帧混合的极端情形), 保留数量最多的尺寸
shape_count = {}
for f in frames:
    shape_count[f.shape] = shape_count.get(f.shape, 0) + 1
dom = max(shape_count, key=shape_count.get)
pairs = [(f, k) for f, k in zip(frames, kept, strict=False) if f.shape == dom]
self.dicom_datasets = [k for _, k in pairs]
self._refresh_patient_info() # 按脱敏状态填患者面板
self.volume_hu = np.array([f for f, _ in pairs])
self.volume_mask = np.zeros_like(self.volume_hu, dtype=np.uint8)
self.global_annotations = {'all': []}
self._hidden_organs.clear() # 换病例时清除上一例的图例隐藏状态
self._mask_undo = []        # 换病例清撤销栈, 防止旧切片号越界访问新蒙版
z, y, x = self.volume_hu.shape
# 默认将 3D 光标定位在体积中心 (中间切片、中间行、中间列)
self.current_3d_pos = [z // 2, y // 2, x // 2]
self.slider_slice.setRange(0, z - 1)
self.slider_slice.setValue(z // 2)
return pid

def _kickoff_ai(self):
    """启动后台 AI 推理。
    每次加载新数据时自增 generation 计数器; 旧 AI 线程回调时若 generation 不匹配则静默
    丢弃结果, 防止旧数据覆盖新数据的蒙版 (竞态条件保护)。
    并作废上一个仍在运行的推理线程, 避免多个 ~8.8GB 推理并发叠加导致内存翻倍/OOM。
    """
    if self.ai_thread is not None and self.ai_thread.isRunning():
        self.ai_thread.cancel()
    self._ai_generation += 1
    gen = self._ai_generation
    self._ai_state = 'running'
    self.lbl_ai_status.setStyleSheet("color: #F1C40F; font-weight: bold;")
    self.lbl_ai_status.setText("Processing AI Pipeline..." if self.is_english else "状态: AI 引擎自动运算中...")
    # lambda 中用 g=gen 捕获当前 generation 值 (闭包变量, 防止后续自增影响比对)
    self.ai_thread = AutoAIEngineThread(
        self.volume_hu,
        callback=lambda mask, t, g=gen: self.on_auto_ai_finished(mask, t, g),
        progress_callback=lambda d, t, g=gen: self.on_ai_progress(d, t, g)
    )
    self.ai_thread.start()

def _on_ai_progress(self, done, total, generation):
    """AI 滑窗推理进度回调 (经 QTimer 投递到主线程)。仅更新当前代的进度显示。"""

```

```

    if generation != self._ai_generation or self._ai_state != 'running':
        return
    pct = int(100 * done / total) if total else 0
    self.lbl_ai_status.setText(
        f"AI Segmenting... {pct}%" if self.is_english else f"状态: AI 分割中... {pct}%"
    )

def _ai_done_text(self):
    """AI 完成后的状态文案: 显示检出的器官类别数 (不含背景与手动追踪)。"""
    n = 0
    if self.volume_mask is not None:
        ids = np.unique(self.volume_mask)
        n = int(((ids != 0) & (ids != MANUAL_TRACK_LABEL)).sum())
    return (f"Ready: {n} organs ({self._ai_time_ms:.0f}ms)" if self.is_english
            else f"状态: 检出 {n} 个器官 ({self._ai_time_ms:.0f}ms)")

def on_auto_ai_finished(self, final_mask, time_ms, generation=None):
    """AI 推理完成的回调 (由 QTimer.singleShot 投递到主线程执行)。

    generation 比对: 防止旧数据的 AI 结果在新数据加载后才回调, 覆盖新数据的蒙版。
    shape 比对: 防止数组维度不匹配导致后续 volume_mask 操作越界。
    recon_mode_active 检查: 若用户已切换到重建实验室, 不触发 update_display,
    避免破坏正在展示的重建结果 (V2/V3/V4 的弦图和重建图像)。
    """
    if generation is not None and generation != self._ai_generation:
        return # 过时的 AI 回调, 静默丢弃
    if self.volume_hu is None or final_mask.shape != self.volume_hu.shape:
        return # 数据已重置或维度不匹配, 安全退出
    self._ai_state = 'done'
    self._ai_time_ms = time_ms
    self.volume_mask = final_mask
    self.lbl_ai_status.setStyleSheet("color: #00FF00; font-weight: bold;")
    self.lbl_ai_status.setText(self._ai_done_text())
    self._update_organ_stats()
    if not self.recon_mode_active:
        self.update_display()

def closeEvent(self, event):
    """关闭窗口: 取消仍在运行的后台 AI 推理, 停止 Cine 定时器。
    动机: AI 单次推理约 8.8GB / ~100s, 关闭窗口若不取消, 线程会继续占内存, 且完成后
    经 Qt 信号回调到已拆除的窗口 (对已删除的 QLabel setText) → RuntimeError。"""
    if self.ai_thread is not None:
        self.ai_thread.cancel()
    self._stop_cine()
    super().closeEvent(event)

def on_slice_changed(self, idx):
    """切片滑动条 valueChanged 回调: 更新 3D 光标 Z 轴坐标并刷新显示。
    无条件 update_display: 临床/对比/重建三种模式各走自己的分支, 确保切片滑条在
    重建实验室也能切换 V1 参考层 (_render_recon_reference 靠 _recon_ref_z 判定是否重置流水线)。"""
    self.current_3d_pos[0] = idx

```

```

self.lbl_slice.setText(f"'{Slice: ' if self.is_english else '层数: '{idx + 1} / {len(self.dicom_datasets)})'")
self.update_display()

def update_display(self):
    """核心显示刷新函数：根据当前模式选择重建实验室分支或临床阅片分支。

    重建实验室模式：仅更新 V1 (参考切片)，并清空 / 禁用 V2-V4 重建流水线。
    临床阅片模式：对每个可见视图按平面切取 2D 截面、做窗宽窗位映射，
    叠加 AI 蒙版、渲染标注、更新 MPR 十字线。
    """
    if self.volume_hu is None:
        return
    z, y, x = self.current_3d_pos

    if self.recon_mode_active:
        self._render_recon_reference(z)
        return

    if self.compare_mode_active and self.compare_volume is not None:
        self._render_compare()
        return

    ww_m, wl_m = self.slider_ww.value(), self.slider_wl.value()
    self.lbl_ww.setText(f"WW: {ww_m}"); self.lbl_wl.setText(f"WL: {wl_m}")
    ds = self.dicom_datasets[z]
    px_sp = self._dcm_float(ds, 'PixelSpacing', 1.0, idx=0)
    # SliceThickness 用于冠/矢状面像素宽高比计算；若缺失/为空则估算为 px_sp*3 (典型螺旋 CT 值)
    slice_thick = self._dcm_float(ds, 'SliceThickness', px_sp * 3)

    for vdata in self.views.values():
        if vdata['container'].isHidden():
            continue
        self._render_clinical_plane(vdata, z, y, x, ww_m, wl_m, px_sp, slice_thick)

    # 图例集中判定：仅当存在可见 Axial 视图开启 Anno 且蒙版有内容时才显示检出器官，
    # 否则清空——避免"关掉 Anno 后叠加已隐藏、图例却仍列着器官"的残留不一致。
    show_legend = self.volume_mask is not None and any(
        vd['plane'] == AXIAL and vd['chk_anno'].isChecked() and not vd['container'].isHidden()
        for vd in self.views.values())
    if show_legend:
        present = np.unique(self.volume_mask[z])
        self._update_legend(present[present != 0])
    else:
        self._update_legend([])

def _patient_display(self):
    """返回用于显示的 (ID, 姓名, 年龄)；脱敏模式下隐去真实身份 (仅显示层, 不改 DICOM)。"""
    if not self.dicom_datasets:
        return ("N/A", "N/A", "")
    if self.anonymize:

```

```

        return ("ANON", "Anonymized" if self.is_english else "已脱敏", "")
    ds = self.dicom_datasets[0]
    pid = str(getattr(ds, 'PatientID', 'N/A'))
    name = str(getattr(ds, 'PatientName', '')) or 'N/A').replace('^', ' ')
    age = str(getattr(ds, 'PatientAge', '') or '')
    return (pid, name, age)

def _refresh_patient_info(self):
    """按当前脱敏状态刷新右侧患者信息面板。"""
    pid, name, age = self._patient_display()
    self.info_labels["ID"].setText(pid)
    self.info_labels["NAME"].setText(name)
    self.info_labels["AGE"].setText(age if age else "N/A")

def _toggle_anonymize(self, on):
    """切换脱敏：刷新患者面板与四角叠加 (overlay 经 update_display 重建)。"""
    self.anonymize = on
    self._refresh_patient_info()
    if not self.recon_mode_active:
        self.update_display()

@staticmethod
def _dcm_float(ds, tag, default, idx=None):
    """安全读取 DICOM 数值标签为 float: 标签缺失、为空 (None)、或无法转 float 时返回 default。
    idx 非空时取序列第 idx 个元素 (如 PixelSpacing[0])。
    动机: getattr 的默认值只在属性【缺失】时生效; 畸形 DICOM 常把数值标签留空
    (pydicom 读回 None), 此时 float(None) / None[idx] 会抛 TypeError, 导致
    加载/显示/定量全线崩溃。此处统一兜底。"""
    v = getattr(ds, tag, None)
    if v is None:
        return default
    try:
        if idx is not None:
            v = v[idx]
        return float(v)
    except (TypeError, ValueError, IndexError):
        return default

@staticmethod
def _safe_name(s, fallback="Unknown"):
    """把患者标识净化为安全文件名片段。患者 DICOM 数据不可信: PatientID/Name
    可能含 '/'、'\'\' 或 '..'，若直接拼进路径会导致存盘失败 (子目录不存在) 甚至
    路径穿越写到导出目录之外。仅保留字母数字/中日韩等词字符与 . _ -, 其余替换为 _。
    再去掉首尾点 (杜绝 '..' 与隐藏文件)，并限长避免 ENAMETOOLONG。
    存/取两侧都经此函数，同一 PatientID 恒定映射到同一文件名，保证往返一致。"""
    s = re.sub(r'[^w.-]', '_', str(s), flags=re.UNICODE).strip('.')
    return (s[:64] or fallback)

def _export_tag(self):
    """导出文件名用的患者标识; 脱敏时返回匿名前缀，防止文件名泄露姓名。"""

```



```

    if self.anonymize or not self.dicom_datasets:
        return "ANON"
    name = str(getattr(self.dicom_datasets[0], 'PatientName', 'P')).replace('^', '_')
    return self._safe_name(name, fallback="P")

def _toggle_overlay(self, on):
    """切换所有视图的 DICOM 信息叠加显隐。"""
    for vd in self.views.values():
        vd['view'].show_overlay = on
        vd['view'].viewport().update()

def _apply_dicom_overlay(self, vdata, plane, z, y, x, ww, wl, px_sp, slice_thick):
    """构建并下发 PACS 风格的四角信息叠加与解剖方位字母。"""
    e = self.is_english
    ds0 = self.dicom_datasets[0]
    Z_MAX, Y_MAX, X_MAX = self.volume_hu.shape
    idx, tot = {AXIAL: (z, Z_MAX), CORONAL: (y, Y_MAX), SAGITTAL: (x, X_MAX)}[plane]
    pname = ({AXIAL: "Axial", CORONAL: "Coronal", SAGITTAL: "Sagittal"} if e else
             {AXIAL: "横断面", CORONAL: "冠状面", SAGITTAL: "矢状面"})[plane]
    zoom = vdata['view'].transform().m11() * 100
    pid, pt_name, age = self._patient_display() # 脱敏时隐去真实身份
    tl = [f"ID: {pid}", pt_name] + ([f"Age: {age}"] if age else [])
    corners = {
        'tl': tl,
        'tr': [f"{getattr(ds0, 'Modality', 'CT')} · V{vdata['view'].view_id}", pname],
        'bl': [f"W: {int(ww)} L: {int(wl)}", f"Zoom: {zoom:.0f}%"],
        'br': [f"{'Slice' if e else '层'} {idx + 1}/{tot}",
               f"Thk {slice_thick:.1f}mm", f"Px {px_sp:.2f}mm"],
    }
    # 解剖方位字母: Axial 图像左=解剖右(R); 冠/矢状面上=头(S)下=足(I)
    orient = {AXIAL: {'top': 'A', 'bottom': 'P', 'left': 'R', 'right': 'L'},
              CORONAL: {'top': 'S', 'bottom': 'I', 'left': 'R', 'right': 'L'},
              SAGITTAL: {'top': 'S', 'bottom': 'I', 'left': 'A', 'right': 'P'}}[plane]
    vdata['view'].set_overlay(corners, orient)

def _render_clinical_plane(self, vdata, z, y, x, ww_m, wl_m, px_sp, slice_thick):
    """临床阅片分支: 渲染单个视图的 2D 截面 + 蒙版 + 标注 + 十字线。"""
    plane = vdata['plane']
    pre = vdata['preset'].currentText()

    # 窗宽/窗位来源: 优先使用各视图独立预设, 否则跟随全局滑动条
    if pre in ["Global", "跟随"]:
        ww, wl = ww_m, wl_m
    else:
        ww, wl = self._WW_PRESETS.get(pre, ww_m), self._WL_PRESETS.get(pre, wl_m)

    # 根据平面切取对应的 2D 截面
    # 像素间距 sp=(行间距, 列间距)=(垂直/Y轴, 水平/X轴), 供 ruler 测距按真实 mm 换算
    # (graphics_view 里  $d=\sqrt{(dx\cdot sp[1])^2+(dy\cdot sp[0])^2}$ ), 故 sp[0] 配垂直、sp[1] 配水平
    if plane == AXIAL:

```

```

        hu = self.volume_hu[z, :, :]
        sp = (px_sp, px_sp) # 横断面: 行/列均为 PixelSpacing
    elif plane == CORONAL:
        hu = self.volume_hu[:, y, :] # (Z, X): 垂直=Z-SliceThickness, 水平=X-PixelSpacing
        sp = (slice_thick, px_sp)
    elif plane == SAGITTAL:
        hu = self.volume_hu[:, :, x] # (Z, Y): 垂直=Z-SliceThickness, 水平=Y-PixelSpacing
        sp = (slice_thick, px_sp)

    # 窗宽窗位映射: HU → [0, 255] 线性映射
    img = np.clip(hu, wl - ww / 2, wl + ww / 2)
    img = ((img - (wl - ww / 2)) / ww * 255).astype(np.uint8)
    if self.chk_invert.isChecked():
        img = 255 - img # 反色 (黑白反转), 观察骨/软组织边界常用
    h, w = img.shape
    qimg = QImage(img.data, w, h, w, QImage.Format_Grayscale8).copy()

    # AI 多器官蒙版叠加: 仅 Axial 平面支持. 用调色板 LUT 一步向量化上色,
    # 每个类别号映射到 constants.LABEL_LUT 中的 RGBA (0=背景为全透明).
    mq = None
    if plane == AXIAL and vdata['chk_anno'].isChecked() and self.volume_mask is not None:
        sm = self.volume_mask[z]
        present = np.unique(sm)
        present = present[present != 0] # 剔除背景, 得到本切片出现的器官类别
        if present.size: # 性能优化: 无器官时跳过 QImage 构建
            lut = LABEL_LUT
            if self._hidden_organs:
                lut = LABEL_LUT.copy()
                for lid in self._hidden_organs:
                    lut[lid] = 0 # 被隐藏器官置为全透明
            ov = np.ascontiguousarray(lut[sm]) # (h,w,4) RGBA
            mq = QImage(ov.data, w, h, w * 4, QImage.Format_RGBA8888).copy()
    # 图例统一在 update_display 末尾按全局状态刷新 (此处不再各视图分别刷, 避免多视图相互覆盖)

    vdata['view'].set_image(QPixmap.fromImage(qimg), mq, sp)
    self._apply_dicom_overlay(vdata, plane, z, y, x, ww, wl, px_sp, slice_thick)
    vdata['view'].clear_annotations() # 清除上一帧的标注图元, 防止重影

    if plane == AXIAL and vdata['chk_anno'].isChecked():
        self._render_annotations(vdata, z, sp)

    # MPR 十字准线: 联动开启时各平面投影不同的坐标轴对
    if self.btn_mpr.isChecked():
        cx, cy = mpr_geometry.voxel_to_crosshair(plane, z, y, x)
        vdata['view'].draw_crosshair(cx, cy)
    else:
        vdata['view'].draw_crosshair(0, 0, show=False)

def select_folder(self):
    """打开文件夹选择对话框, 选择后触发 DICOM 加载."""

```

```

        p = QFileDialog.getExistingDirectory(self, "Select Folder")
        if p:
            self.load_data(p)

# =====
# 矩阵重建共用工具
# =====
def main():
    """程序入口。可选 --data DIR: 启动即加载该 DICOM 目录（默认不加载任何数据）。"""
    import argparse
    import multiprocessing

    # freeze_support: 多进程 'spawn' 模式在 macOS/Windows 打包环境中必须调用,
    # 防止子进程重入主程序逻辑导致无限递归启动
    multiprocessing.freeze_support()
    parser = argparse.ArgumentParser(description="医学影像工作站 Pro + 重建实验室")
    parser.add_argument("--data", metavar="DIR", default=None,
                        help="启动时加载的 DICOM 目录（可选，默认不加载任何数据）")
    args, qt_args = parser.parse_known_args()
    app = QApplication(sys.argv[1:] + qt_args)
    window = MedicalViewer(data_dir=args.data)
    window.show()
    sys.exit(app.exec())

if __name__ == "__main__":
    main()

# ===== ui_builder.py (396 行) =====
# =====
# UI 构建 Mixin
# 负责: 主窗口三栏布局与全部控件的构建 (左工具栏 / 中视图栅格 / 右控制面板 /
#       临床阅片 Tab / 重建实验室 Tab / 单个联动视图 / 暗色主题)。
#
# 设计: 以 Mixin 形式并入 MedicalViewer, 在 __init__ 中调用 setup_stylesheet()
#       与 init_ui() 完成装配。这些方法创建 self.xxx 控件并把信号连到留在
#       main / 各 Mixin 的槽方法上, 全部经 self 在合并实例上解析。仅负责"搭建",
#       运行期逻辑 (update_language 重译、change_view_plane、switch_layout 等) 仍在 main。
# =====

import os

from PySide6.QtCore import Qt
from PySide6.QtWidgets import (
    QButtonGroup,
    QCheckBox,
    QComboBox,
    QFormLayout,
    QFrame,
    QGridLayout,
    QGroupBox,

```

```
        QHBoxLayout,
        QLabel,
        QPushButton,
        QRadioButton,
        QSizePolicy,
        QSlider,
        QSpinBox,
        QSplitter,
        QTabWidget,
        QVBoxLayout,
        QWidget,
    )

from constants import AXIAL, MANUAL_TRACK_LABEL
from graphics_view import MedicalGraphicsView

class UiBuilderMixin:
    """主窗口 UI 构建方法集合, 混入 MedicalViewer。"""

    def setup_stylesheet(self):
        """从 style.qss 加载暗色主题样式表; 文件缺失时静默跳过, UI 仍可用 Qt 默认样式渲染。"""
        qss_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), "style.qss")
        try:
            with open(qss_path, encoding='utf-8') as f:
                self.setStyleSheet(f.read())
        except OSError as e:
            print(f"Warning: failed to load style.qss: {e}")

    def init_ui(self):
        """构建主窗口三栏布局: 左工具栏 | 中视图栅格 | 右控制面板。"""
        mw = QWidget()
        self.setCentralWidget(mw)
        l = QHBoxLayout(mw); l.setContentsMargins(0, 0, 0, 0); l.setSpacing(0)

        self._build_left_toolbar()
        self._build_view_grid()
        self._build_right_panel()

        l.addWidget(self.left_toolbar)
        l.addWidget(self.main_splitter, 1)
        l.addWidget(self.right_panel)

    def _build_left_toolbar(self):
        """左侧 70px 宽工具按钮列: 指针/卡尺/画笔/矩形/套索/3D 追踪, 互斥选中。"""
        self.left_toolbar = QFrame()
        self.left_toolbar.setObjectName("LeftToolbar")
        self.left_toolbar.setFixedWidth(70)
        ll = QVBoxLayout(self.left_toolbar); ll.setContentsMargins(5, 20, 5, 20); ll.setSpacing(15)
```

```

self.tool_btn_group = QButtonGroup(self)
self.tool_btns = {}
tool_data = [(0, 'btn_ptr'), (1, 'btn_rul'), (2, 'btn_drw'),
              (4, 'btn_rec'), (3, 'btn_las'), (5, 'btn_trk'),
              (6, 'btn_brush'), (7, 'btn_erase'), # 6/7=分割修正: 画笔补画 / 橡皮擦除
              (8, 'btn_roi')] # 8=椭圆 ROI 密度测量
for tid, key in tool_data:
    b = QPushButton(); b.setProperty("class", "ToolBtn"); b.setCheckable(True); b.setChecked(tid == 0)
    self.tool_btn_group.addButton(b, tid); ll.addWidget(b); self.tool_btns[key] = b
self.tool_btn_group.idClicked.connect(self.change_active_tool)
ll.addStretch()

def _build_view_grid(self):
    """中央 4 视图栅格: QSplitter 嵌套结构 (main_splitter 含 top/bottom 两个横向 splitter)。"""
    self.main_splitter = QSplitter(Qt.Vertical)
    self.top_splitter = QSplitter(Qt.Horizontal)
    self.bottom_splitter = QSplitter(Qt.Horizontal)
    for vid in (1, 2, 3, 4):
        self.create_independent_view(vid, AXIAL)
        self.top_splitter.addWidget(self.views[1]['container'])
        self.top_splitter.addWidget(self.views[2]['container'])
        self.bottom_splitter.addWidget(self.views[3]['container'])
        self.bottom_splitter.addWidget(self.views[4]['container'])
    self.main_splitter.addWidget(self.top_splitter)
    self.main_splitter.addWidget(self.bottom_splitter)

def _build_right_panel(self):
    """右侧 320px 宽控制面板: 语言切换 / 加载 / 保存 + 两个 Tab (临床阅片 / 重建实验室)。"""
    self.right_panel = QFrame()
    self.right_panel.setObjectName("RightPanel")
    self.right_panel.setFixedWidth(320)
    rl = QVBoxLayout(self.right_panel); rl.setContentsMargins(12, 12, 12, 12); rl.setSpacing(5)

    # 顶部: 语言切换按钮 (靠右)
    th = QHBoxLayout()
    self.btn_lang = QPushButton("EN"); self.btn_lang.setFixedWidth(40)
    self.btn_lang.setStyleSheet("font-size: 10px; color: #5C677D; border: 1px solid #373E4D;")
    self.btn_lang.clicked.connect(self.toggle_language)
    th.addStretch(); th.addWidget(self.btn_lang); rl.addLayout(th)

    self.btn_import = QPushButton("加载 DICOM 目录"); self.btn_import.setObjectName("PrimaryBtn")
    self.btn_import.clicked.connect(self.select_folder); rl.addWidget(self.btn_import)
    self.btn_save_proj = QPushButton("保存标注工程"); self.btn_save_proj.setProperty("class", "ActionBtn")
    self.btn_save_proj.clicked.connect(self.save_project); rl.addWidget(self.btn_save_proj)

    self.tabs = QTabWidget()
    self.tab_clinical = QWidget()
    self.tab_recon = QWidget()
    self.tabs.addTab(self.tab_clinical, "临床阅片")
    self.tabs.addTab(self.tab_recon, "重建实验室")

```

```

self._build_clinical_tab()
self._build_recon_tab()
# 信号连接必须在两个 tab 构建完成后: 否则 addTab 触发的 currentChanged 会
# 在 btn_dfr 等重建控件尚未创建时调用 on_tab_changed, 抛 AttributeError
self.tabs.currentChanged.connect(self.on_tab_changed)
rl.addWidget(self.tabs)

def _build_clinical_tab(self):
    """临床阅片 Tab: 患者信息 / 显示控制 (布局+MPR+三滑条+预设) / AI 状态 / 测量与清理。"""
    t1_layout = QVBoxLayout(self.tab_clinical)
    t1_layout.setContentsMargins(0, 0, 0, 0)

    # 患者信息分组
    self.grp_patient = QGroupBox("患者信息")
    info_layout = QFormLayout(); info_layout.setContentsMargins(10, 15, 10, 10)
    self.info_labels = {"ID": QLabel("N/A"), "NAME": QLabel("N/A"), "AGE": QLabel("N/A")}
    for k, v in self.info_labels.items():
        v.setObjectName("ValueText"); info_layout.addRow(QLabel(k), v)
    self.grp_patient.setLayout(info_layout)
    t1_layout.addWidget(self.grp_patient)

    # 显示控制分组 (布局下拉、MPR 按钮、三滑条、预设窗口栅格)
    self.grp_display = QGroupBox("显示控制")
    dl = QVBoxLayout(); dl.setContentsMargins(10, 15, 10, 10)
    top_dl = QHBoxLayout()
    self.combo_layout = QComboBox()
    self.combo_layout.currentIndexChanged.connect(self.switch_layout)
    self.combo_layout.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
    top_dl.addWidget(self.combo_layout)
    self.btn_mpr = QPushButton("MPR 联动: 关"); self.btn_mpr.setObjectName("MprBtn"); self.btn_mpr.setCheckable(True)
    self.btn_mpr.clicked.connect(self.on_mpr_toggled)
    self.btn_mpr.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
    top_dl.addWidget(self.btn_mpr)
    top_dl.setStretch(0, 3); top_dl.setStretch(1, 2)
    dl.addLayout(top_dl)

    self.lbl_slice = QLabel(); self.slider_slice = QSlider(Qt.Horizontal)
    self.slider_slice.valueChanged.connect(self.on_slice_changed)
    self.lbl_wv = QLabel(); self.slider_wv = QSlider(Qt.Horizontal)
    self.slider_wv.setRange(1, 4000); self.slider_wv.setValue(1500); self.slider_wv.valueChanged.connect(self.update_display)
    self.lbl_wl = QLabel(); self.slider_wl = QSlider(Qt.Horizontal)
    self.slider_wl.setRange(-1200, 1200); self.slider_wl.setValue(-500); self.slider_wl.valueChanged.connect(self.update_display)
    for lbl, slider in [(self.lbl_slice, self.slider_slice), (self.lbl_wv, self.slider_wv), (self.lbl_wl, self.slider_wl)]:
        lbl.setFixedWidth(76); row = QHBoxLayout(); row.setSpacing(6); row.addWidget(lbl); row.addWidget(slider); dl.addLayout(row)
    self.lbl_wv_hint = QLabel(); self.lbl_wv_hint.setStyleSheet("color: #5C677D; font-size: 10px;")
    dl.addWidget(self.lbl_wv_hint)

    # 6 个临床预设窗口按钮 (3 列栅格)
    pl = QGridLayout(); self.preset_btns = []
    for i, (n, ww, wl) in enumerate([("Lung", 1500, -500), ("Medi", 400, 40), ("Bone", 1500, 400),

```

```

        ("Vasc", 600, 150), ("Abdo", 150, 30), ("Brain", 80, 40})):
        b = QPushButton(n); b.setProperty("class", "ActionBtn"); b.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
        b.clicked.connect(lambda c, w=ww, l=wl: self.set_window(w, l))
        pl.addWidget(b, i // 3, i % 3); self.preset_btns.append(b)
    pl.setColumnStretch(0, 1); pl.setColumnStretch(1, 1); pl.setColumnStretch(2, 1)
    dl.addLayout(pl)
    # 信息叠加显隐 + 反色 (医学影像软件标配)
    h_disp = QHBoxLayout()
    self.chk_overlay = QCheckBox("信息叠加"); self.chk_overlay.setChecked(True)
    self.chk_overlay.toggled.connect(self._toggle_overlay)
    self.chk_invert = QCheckBox("反色"); self.chk_invert.toggled.connect(self.update_display)
    self.chk_anon = QCheckBox("脱敏"); self.chk_anon.toggled.connect(self._toggle_anonymize)
    h_disp.addWidget(self.chk_overlay); h_disp.addWidget(self.chk_invert); h_disp.addWidget(self.chk_anon)
    dl.addLayout(h_disp)
    # Cine 电影播放 (往返连播, 速度可调) + 双序列随访对比
    h_nav = QHBoxLayout()
    self.btn_cine = QPushButton("▶ 播放"); self.btn_cine.setProperty("class", "ActionBtn")
    self.btn_cine.clicked.connect(self.toggle_cine)
    self.cb_cine_speed = QComboBox() # 播放速度: itemData 为定时器间隔(ms)
    self.cb_cine_speed.addItem("慢", 250); self.cb_cine_speed.addItem("中", 120); self.cb_cine_speed.addItem("快", 50)
    self.cb_cine_speed.setCurrentIndex(1)
    self.cb_cine_speed.currentIndexChanged.connect(self.on_cine_speed_changed)
    self.btn_compare = QPushButton("加载对比序列"); self.btn_compare.setProperty("class", "ActionBtn")
    self.btn_compare.clicked.connect(self.toggle_compare)
    h_nav.addWidget(self.btn_cine); h_nav.addWidget(self.cb_cine_speed); h_nav.addWidget(self.btn_compare)
    dl.addLayout(h_nav)
    self.grp_display.setLayout(dl)
    ti_layout.addWidget(self.grp_display)

    # AI 状态分组 (原 AI 按钮改为状态显示, 因为已全自动)
    self.grp_ai = QGroupBox("自动化 AI 引擎")
    ai_layout = QVBoxLayout(); ai_layout.setContentsMargins(10, 15, 10, 10)
    self.lbl_ai_status = QLabel("状态: 待机中")
    self.lbl_ai_status.setStyleSheet("color: #8B949E; font-weight: bold;")
    ai_layout.addWidget(self.lbl_ai_status)
    # 器官图例: 显示当前切片检测到的器官及其颜色 (随切片刷新)
    self.lbl_ai_legend = QLabel("")
    self.lbl_ai_legend.setWordWrap(True)
    self.lbl_ai_legend.setTextFormat(Qt.RichText)
    self.lbl_ai_legend.setStyleSheet("font-size: 11px;")
    # 图例条目做成可点击链接: 单击切换该器官在蒙版叠加中的显隐
    self.lbl_ai_legend.setTextInteractionFlags(Qt.LinksAccessibleByMouse)
    self.lbl_ai_legend.linkActivated.connect(self._toggle_organ)
    ai_layout.addWidget(self.lbl_ai_legend)
    # 器官定量: 分割完成后列出各器官体积/平均 HU, 并支持导出 CSV
    self.lbl_ai_stats = QLabel("")
    self.lbl_ai_stats.setWordWrap(True)
    self.lbl_ai_stats.setTextFormat(Qt.RichText)
    self.lbl_ai_stats.setStyleSheet("font-size: 11px; color: #B0B8C4;")
    ai_layout.addWidget(self.lbl_ai_stats)

```

```

self.btn_export_stats = QPushButton("导出定量 CSV")
self.btn_export_stats.setProperty("class", "ActionBtn")
self.btn_export_stats.setEnabled(False)
self.btn_export_stats.clicked.connect(self.export_organ_stats)
ai_layout.addWidget(self.btn_export_stats)
# 合规免责声明: AI 结果非诊断依据, 常驻显示
self.lbl_disclaimer = QLabel("△ AI 结果与器官标签为自动推断, 仅供参考, 非诊断依据。")
self.lbl_disclaimer.setWordWrap(True)
self.lbl_disclaimer.setStyleSheet("color: #C0392B; font-size: 10px;")
ai_layout.addWidget(self.lbl_disclaimer)
self.grp_ai.setLayout(ai_layout)
t1_layout.addWidget(self.grp_ai)

# 测量与清理分組
self.grp_measure = QGroupBox("测量与清理")
ml = QVBoxLayout(); ml.setContentsMargins(10, 15, 10, 10)
# 光标 HUD: 鼠标悬停时实时显示 (x,y,z) 坐标 / HU 值 / 所在器官
self.lbl_hud = QLabel("")
self.lbl_hud.setStyleSheet("color: #8B949E; font-family: monospace; font-size: 11px; min-height: 16px; max-height: 16px;")
self.lbl_hud.setAlignment(Qt.AlignCenter); ml.addWidget(self.lbl_hud)
self.lbl_hu_value = QLabel()
self.lbl_hu_value.setStyleSheet("color: #00ADB5; font-weight: bold; font-size: 13px; min-height: 18px; max-height: 18px;")
self.lbl_hu_value.setAlignment(Qt.AlignCenter); ml.addWidget(self.lbl_hu_value)
self.chk_global_scope = QCheckBox("新标注穿透所有切片"); ml.addWidget(self.chk_global_scope)
# 分割修正画笔半径 (画笔/橡皮工具共用)
h_brush = QHBoxLayout()
self.lbl_brush = QLabel("画笔半径:")
self.spin_brush = QSpinBox(); self.spin_brush.setRange(1, 40); self.spin_brush.setValue(6)
self.spin_brush.valueChanged.connect(self._set_brush_radius)
h_brush.addWidget(self.lbl_brush); h_brush.addWidget(self.spin_brush)
ml.addLayout(h_brush)
# 画笔目标: 补画写入哪个标签 (手动标注 or 当前检出的某个器官, 使修正计入该器官定量)
h_target = QHBoxLayout()
self.lbl_paint_target = QLabel("画笔目标:")
self.cb_paint_target = QComboBox()
self.cb_paint_target.addItem("手动标注", MANUAL_TRACK_LABEL)
h_target.addWidget(self.lbl_paint_target); h_target.addWidget(self.cb_paint_target)
ml.addLayout(h_target)
self.btn_clear_anno = QPushButton("清空蒙版与标注"); self.btn_clear_anno.setProperty("class", "ActionBtn")
self.btn_clear_anno.clicked.connect(self.clear_current_slice_annotations); ml.addWidget(self.btn_clear_anno)
self.btn_reset = QPushButton("重置工作区"); self.btn_reset.setObjectName("DangerBtn")
self.btn_reset.clicked.connect(self.reset_all_states); ml.addWidget(self.btn_reset)
self.grp_measure.setLayout(ml)
t1_layout.addWidget(self.grp_measure)
t1_layout.addStretch()

def _build_recon_tab(self):
    """重建实验室 Tab: 投影生成 / BP-FBP-DFR / DMR-ART-SIRT / 性能监控。"""
    t2_layout = QVBoxLayout(self.tab_recon)
    t2_layout.setContentsMargins(0, 0, 0, 0)

```



```
# 投影生成分组: 角度单选 + 生成按钮
self.grp_proj = QGroupBox("X射线投影生成")
play = QVBoxLayout(); play.setSpacing(10)
self.rad_60 = QRadioButton("60°"); self.rad_120 = QRadioButton("120°")
self.rad_180 = QRadioButton("180°"); self.rad_360 = QRadioButton("360°")
self.rad_180.setChecked(True)
h_rad = QHBoxLayout()
for r in [self.rad_60, self.rad_120, self.rad_180, self.rad_360]:
    r.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed); h_rad.addWidget(r)
h_rad.setStretch(0, 1); h_rad.setStretch(1, 1); h_rad.setStretch(2, 1); h_rad.setStretch(3, 1)
play.addLayout(h_rad)
# 采样密度: 在选定角度范围内的投影数量倍率。越高角度间隔越细、重建质量越好 (越慢)。
h_dens = QHBoxLayout()
self.lbl_oversample = QLabel("采样密度:")
self.combo_oversample = QComboBox()
self.combo_oversample.addItem("标准 1x", "高 2x", "超高 4x")
self.combo_oversample.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
h_dens.addWidget(self.lbl_oversample); h_dens.addWidget(self.combo_oversample)
play.addLayout(h_dens)
self.btn_gen_sino = QPushButton("发射射线生成弦图"); self.btn_gen_sino.setProperty("class", "ActionBtn")
self.btn_gen_sino.setStyleSheet("background-color: #D35400; color: white;")
self.btn_gen_sino.clicked.connect(self.generate_sinogram)
play.addWidget(self.btn_gen_sino)
self.grp_proj.setLayout(play)
t2_layout.addWidget(self.grp_proj)

# 图像重建算法分组: DFR / BP / 滤波器选择 / FBP
self.grp_algo = QGroupBox("图像重建算法")
alay = QVBoxLayout(); alay.setSpacing(10)
self.btn_dfr = QPushButton("直接傅里叶重建 (DFR)"); self.btn_dfr.setProperty("class", "ActionBtn")
self.btn_dfr.clicked.connect(self.run_dfr)
self.btn_bp = QPushButton("反投影法 (BP - 未滤波)"); self.btn_bp.setProperty("class", "ActionBtn")
self.btn_bp.clicked.connect(self.run_bp)

h_fbp = QHBoxLayout()
self.lbl_filter_text = QLabel("选择滤波器:"); self.lbl_filter_text.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
h_fbp.addWidget(self.lbl_filter_text)
self.cb_filter = QComboBox()
self.cb_filter.addItem("Ram-Lak", "Shepp-Logan", "Cosine", "Hamming", "Hann")
self.cb_filter.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
h_fbp.addWidget(self.cb_filter)
h_fbp.setStretch(0, 1); h_fbp.setStretch(1, 2)
alay.addLayout(h_fbp)

self.btn_fbp = QPushButton("滤波反投影 (FBP) 对比"); self.btn_fbp.setProperty("class", "ActionBtn")
self.btn_fbp.setStyleSheet("background-color: #27AE60; color: white;")
self.btn_fbp.clicked.connect(self.run_fbp)

alay.addWidget(self.btn_dfr); alay.addWidget(self.btn_bp); alay.addWidget(self.btn_fbp)
```

```

self.grp_algo.setLayout(alay)
t2_layout.addWidget(self.grp_algo)
# DFR/BP/FBP 三个重建按钮在生成弦图前保持禁用, 强制工作流程顺序: 先投影再重建
for b in [self.btn_dfr, self.btn_bp, self.btn_fbp]:
    b.setEnabled(False)

# 矩阵重建分组: 尺寸 / 方法 / 迭代次数 / DMR / ART 按钮
self.grp_matrix = QGroupBox("直接矩阵重建 & ART / SIRT")
m_lay = QVBoxLayout(); m_lay.setSpacing(8)
h_ms = QHBoxLayout()
self.lbl_matrix_size = QLabel("图像尺寸:"); self.lbl_matrix_size.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
self.cb_matrix_size = QComboBox(); self.cb_matrix_size.addItem("16×16"); self.cb_matrix_size.addItem("32×32"); self.cb_matrix_size.addItem("64×64")
self.cb_matrix_size.setCurrentIndex(1); self.cb_matrix_size.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
h_ms.addWidget(self.lbl_matrix_size); h_ms.addWidget(self.cb_matrix_size)
h_ms.setStretch(0, 1); h_ms.setStretch(1, 2); m_lay.addLayout(h_ms)
h_mm = QHBoxLayout()
self.lbl_art_method = QLabel("迭代方法:"); self.lbl_art_method.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
self.cb_art_method = QComboBox(); self.cb_art_method.addItem("ART"); self.cb_art_method.addItem("SIRT")
self.cb_art_method.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
h_mm.addWidget(self.lbl_art_method); h_mm.addWidget(self.cb_art_method)
h_mm.setStretch(0, 1); h_mm.setStretch(1, 2); m_lay.addLayout(h_mm)
h_mi = QHBoxLayout()
self.lbl_art_iter = QLabel("迭代次数:"); self.lbl_art_iter.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
self.cb_art_iter = QComboBox(); self.cb_art_iter.addItem("10"); self.cb_art_iter.addItem("20"); self.cb_art_iter.addItem("50")
self.cb_art_iter.setCurrentIndex(1); self.cb_art_iter.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
h_mi.addWidget(self.lbl_art_iter); h_mi.addWidget(self.cb_art_iter)
h_mi.setStretch(0, 1); h_mi.setStretch(1, 2); m_lay.addLayout(h_mi)
self.btn_dmr = QPushButton("直接矩阵重建 (DMR)"); self.btn_dmr.setProperty("class", "ActionBtn")
self.btn_dmr.setStyleSheet("background-color: #1A5276; color: white;"); self.btn_dmr.clicked.connect(self.run_dmr)
self.btn_art = QPushButton("ART / SIRT 迭代重建"); self.btn_art.setProperty("class", "ActionBtn")
self.btn_art.setStyleSheet("background-color: #145A32; color: white;"); self.btn_art.clicked.connect(self.run_art_sirt)
m_lay.addWidget(self.btn_dmr); m_lay.addWidget(self.btn_art)
self.grp_matrix.setLayout(m_lay)
t2_layout.addWidget(self.grp_matrix)
# DMR/ART 不依赖弦图 (自行生成小图并计算), 但需要有 DICOM 数据才能运行
for b in [self.btn_dmr, self.btn_art]:
    b.setEnabled(False)

# 性能监控分组: 耗时显示
self.grp_mon = QGroupBox("算法性能监控")
m_lay = QVBoxLayout()
self.lbl_time = QLabel("运行耗时: -- ms")
self.lbl_time.setStyleSheet("color: #00FF00; font-family: monospace; font-size: 14px; font-weight: bold; background-color: #000000; padding: 6px; border-radius: 4px; border: 1px solid #333; min-height: 20px; max-height: 20px;")
self.lbl_time.setAlignment(Qt.AlignCenter)
m_lay.addWidget(self.lbl_time)
self.grp_mon.setLayout(m_lay)
t2_layout.addWidget(self.grp_mon)
t2_layout.addStretch()

```

```

def create_independent_view(self, vid, plane=AXIAL):
    c = QFrame(); c.setObjectName("ViewContainer"); lay = QVBoxLayout(c); lay.setContentsMargins(0,0,0,0); lay.setSpacing(0)
    t = QFrame(); t.setObjectName("ViewToolBar"); t.setFixedHeight(32)
    tl = QHBoxLayout(t); tl.setContentsMargins(8,2,8,2); tl.setSpacing(6)
    lt = QLabel(f"V{vid}"); lt.setStyleSheet("color: #C9D1D9; font-weight: bold; min-width: 20px;")
    cb_plane = QComboBox(); cb_plane.setFixedWidth(80)
    ps = QComboBox(); ps.setFixedWidth(85); ps.currentIndexChanged.connect(self.update_display)
    an = QCheckBox(); an.setObjectName("ViewOption"); an.setChecked(True); an.stateChanged.connect(self.update_display)
    lk = QCheckBox(); lk.setObjectName("ViewOption"); lk.stateChanged.connect(self.update_display)
    tl.addWidget(lt); tl.addWidget(cb_plane); tl.addWidget(ps); tl.addStretch(); tl.addWidget(an); tl.addWidget(lk)
    v = MedicalGraphicsView(vid)
    v.clicked_pos.connect(lambda p, id=vid: self.measure_hu(p, id))
    v.wheel_scrolled.connect(lambda d, id=vid: self.on_wheel_mpr(d, id))
    v.annotation_added.connect(self.handle_annotation_added)
    v.annotation_deleted.connect(self.handle_annotation_deleted)
    v.crop_requested.connect(lambda pts, id=vid: self.handle_crop_requested(id, pts))
    v.track_requested.connect(lambda r, id=vid: self.handle_3d_track_requested(id, r))
    v.window_changed.connect(self.on_window_changed_by_mouse)
    v.mouse_hovered.connect(lambda pos, id=vid: self.sync_crosshair(pos, id))
    v.seg_paint_requested.connect(lambda pts, er, id=vid: self.handle_seg_paint(id, pts, er))
    lay.addWidget(t); lay.addWidget(v); t.raise_()
    self.views[vid] = {'container':c, 'cb_plane': cb_plane, 'preset':ps, 'lock':lk, 'chk_anno':an, 'view':v, 'plane': plane, 'title_label': l
t}

    cb_plane.currentIndexChanged.connect(lambda idx, v_id=vid: self.change_view_plane(v_id, idx))

# ===== interaction.py (148 行) =====
# =====
# 交互 Mixin: Cine 电影播放 + MPR 联动 / 导航
# 负责: MPR 十字线联动 (平面切换、悬停同步、滚轮换层/换面、HU 探针、光标 HUD)
# 与 Cine 电影播放 (往返连播、调速、启停)。
#
# 设计: 以 Mixin 形式并入 MedicalViewer。方法经 self 访问主窗口控件与状态
# (self.views / self.slider_slice / self.cine_timer / self.current_3d_pos 等)
# 及 update_display 等留在 main 的方法。
# 注意: keyPressEvent 是 QMainWindow 的重写方法, 因 MRO 中 QMainWindow 先于
# Mixin, 若放到 Mixin 会被遮蔽, 故留在 MedicalViewer 本体。
# =====

from PySide6.QtCore import Qt, QTimer

import mpr_geometry
from constants import AXIAL, CORONAL, SAGITTAL, TOOL_POINTER

class InteractionMixin:
    """Cine 播放与 MPR 联动 / 导航交互方法集合, 混入 MedicalViewer。"""

    def on_mpr_toggled(self, checked):
        self.update_language()
        if checked:
            default_planes = [0, AXIAL, CORONAL, SAGITTAL, AXIAL]

```

```

        for vid, v in self.views.items(): v['cb_plane'].setCurrentIndex(default_planes[vid])
        self.update_display()
    else:
        for vdata in self.views.values(): vdata['view'].draw_crosshair(0, 0, show=False)

def change_view_plane(self, vid, plane_idx):
    """切换某个视图的成像平面（横断/冠状/矢状）。
    切换后延迟 20ms 做 fitInView，等待新图像渲染完成后再适配缩放，
    避免基于旧图像尺寸计算比例。
    """
    if plane_idx < 0:
        return # 下拉框清空重填时会触发 index=-1, 需要过滤
    self.views[vid]['plane'] = plane_idx
    if not self.recon_mode_active:
        self.update_display()
    v = self.views[vid]['view']
    QTimer.singleShot(20, lambda: v.fitInView(v.scene.sceneRect(), Qt.KeepAspectRatio))

def sync_crosshair(self, scene_pos, vid):
    """MPR 联动：当用户在任意视图中移动鼠标时，同步更新所有视图的十字准线位置。

    坐标映射规则（三平面共用同一个 3D 光标 [z, y, x]）:
    - Axial 视图 → 鼠标 (px, py) 对应 3D 的 (x=px, y=py), z 不变
    - Coronal 视图 → 鼠标 (px, py) 对应 3D 的 (x=px, z=py), y 不变
    - Sagittal 视图 → 鼠标 (px, py) 对应 3D 的 (y=px, z=py), x 不变
    """
    if self.volume_hu is None or self.recon_mode_active or self.compare_mode_active:
        return
    source_plane = self.views[vid]['plane']
    pos_x, pos_y = int(scene_pos.x()), int(scene_pos.y())
    # 悬停像素 → 完整 3D 体素（非 source 平面轴沿用当前光标），并裁剪到体积范围
    z, y, x = mpr_geometry.hover_to_voxel(source_plane, pos_x, pos_y,
                                          tuple(self.current_3d_pos), self.volume_hu.shape)
    self._update_hud(z, y, x) # HUD 实时更新，不依赖 MPR 联动开关
    if not self.btn_mpr.isChecked():
        return
    self.current_3d_pos = [z, y, x]
    # 同步切片滑条（blockSignals 防止递归触发 on_slice_changed），保持三者一致
    if self.slider_slice.value() != z:
        self.slider_slice.blockSignals(True)
        self.slider_slice.setValue(z)
        self.slider_slice.blockSignals(False)
        self.lbl_slice.setText(f"'Slice: ' if self.is_english else '层数: '){z + 1} / {len(self.dicom_datasets)}")
    # 刷新所有平面图像到新光标位置（真正的 MPR 联动，而非仅移动十字线）
    self.update_display()
    # 十字线须在 update_display 之后重画（set_image 会重置线段坐标）
    for vdata in self.views.values():
        if vdata['container'].isHidden():
            continue
        cx, cy = mpr_geometry.voxel_to_crosshair(vdata['plane'], z, y, x)

```

```

        vdata['view'].draw_crosshair(cx, cy)

def _update_hud(self, z, y, x):
    """更新光标 HUD: 显示 (x,y,z) 坐标、该体素 HU 值、以及所在器官 (若有分割)。"""
    hu = float(self.volume_hu[z, y, x])
    txt = f"({x}, {y}, {z}) {hu:.0f} HU"
    if self.volume_mask is not None:
        lid = int(self.volume_mask[z, y, x])
        if lid != 0:
            name = self.organ_names.get(lid, ("", ""))[1 if self.is_english else 0]
            if name:
                txt += f" · {name}"
    self.lbl_hud.setText(txt)

def toggle_cine(self):
    """开始/停止 Cine 电影播放 (自动连续翻片, 到末层循环回第一层)。"""
    if self.volume_hu is None or self.recon_mode_active:
        return
    if self.cine_timer.isActive():
        self._stop_cine()
    else:
        self.cine_timer.start(int(self.cb_cine_speed.currentData()))
        self.btn_cine.setText("⏸ Pause" if self.is_english else "⏸ 暂停")

def _on_cine_speed_changed(self):
    """播放中改速度即时生效。"""
    if self.cine_timer.isActive():
        self.cine_timer.start(int(self.cb_cine_speed.currentData()))

def _cine_step(self):
    """Cine 定时器回调: 往返(bounce)翻片—到顶/到底反向, 避免跳变回环。"""
    if self.volume_hu is None or self.recon_mode_active:
        self._stop_cine(); return
    mx = self.slider_slice.maximum()
    nz = self.slider_slice.value() + self._cine_dir
    if nz > mx:
        nz, self._cine_dir = mx - 1, -1
    elif nz < 0:
        nz, self._cine_dir = 1, 1
    self.slider_slice.setValue(max(0, min(nz, mx)))

def _stop_cine(self):
    """停止 Cine 播放并复位按钮文案。"""
    self.cine_timer.stop()
    self.btn_cine.setText("▶ Play" if self.is_english else "▶ 播放")

def on_wheel_mpr(self, d, vid):
    if self.volume_hu is None or self.recon_mode_active: return

```

```

        increment = -1 if d > 0 else 1
        Z_MAX, Y_MAX, X_MAX = self.volume_hu.shape; z, y, x = self.current_3d_pos
        if self.compare_mode_active: # 对比模式: 滚轮始终换主序列切片, V2 按比例联动
            self.slider_slice.setValue(max(0, min(z + increment, Z_MAX - 1)))
            return
        plane = self.views[vid]['plane']
        if plane == AXIAL: self.slider_slice.setValue(max(0, min(z + increment, Z_MAX - 1)))
        elif plane == CORONAL: self.current_3d_pos[1] = max(0, min(y + increment, Y_MAX - 1)); self.update_display()
        elif plane == SAGITTAL: self.current_3d_pos[2] = max(0, min(x + increment, X_MAX - 1)); self.update_display()

    def measure_hu(self, p, vid):
        if self.active_tool == TOOL_POINTER and self.volume_hu is not None and not self.recon_mode_active and not self.compare_mode_active:
            vd = self.views.get(vid); c = vd['view'].get_real_coordinates(p); plane = vd['plane']
            if c:
                try:
                    if plane == AXIAL: val = self.volume_hu[self.current_3d_pos[0], c[1], c[0]]
                    elif plane == CORONAL: val = self.volume_hu[c[1], self.current_3d_pos[1], c[0]]
                    elif plane == SAGITTAL: val = self.volume_hu[c[1], c[0], self.current_3d_pos[2]]
                    plane_str = {"Axial": "Axial", "Coronal": "Coronal", "Sagittal": "Sagittal"} if self.is_english else {"Axial": "横断面", "Coron
al": "冠状面", "Sagittal": "矢状面"}
                    p_name = plane_str.get({AXIAL: "Axial", CORONAL: "Coronal", SAGITTAL: "Sagittal"}[plane])
                    self.lbl_hu_value.setText(f"V{vid} [{p_name}] ({c[0]}, {c[1]}) : {val:.1f} HU")
                except Exception: pass

# ===== recon_lab.py (520 行) =====
# =====
# 重建实验室 Mixin
# 负责: CT 断层重建教学实验室的全部 UI 调度 (投影生成 / BP / FBP / DFR /
#       DMR / ART / SIRT, 重建模式进出、视图刷新)。
#
# 设计: 以 Mixin 形式并入 MedicalViewer (class MedicalViewer(QMainWindow,
#       ReconLabMixin))。这些方法通过 self 访问主窗口的 UI 控件与状态
#       (self.views / self.slider_* / self.btn_* / self.volume_hu 等),
#       以及留在 main.py 的共享方法 (self.set_view_title / self.update_display /
#       self._apply_grid_visibility / self._apply_grid_sizes)。
#       纯数值计算仍在 recon.py (recon_lib), 本模块只做 UI 接线与调度。
# =====

import time

import numpy as np
from PySide6.QtCore import Qt, QTimer
from PySide6.QtGui import QImage, QPainter, QPixmap
from PySide6.QtWidgets import QApplication, QMessageBox, QProgressDialog

import recon as recon_lib

class ReconLabMixin:
    """重建实验室相关方法集合, 混入 MedicalViewer。"""

```

```
        # 防跑出画面：右侧放不下则移到椭圆左侧，纵向夹取在图像内
        tx = rx0 + rw + 4
        if tx + 95 > W:
            tx = max(0, rx0 - 95)
        ty = min(max(0.0, ry0), max(0.0, H - 46))
        txt.setPos(tx, ty)
        vdata['view'].scene.addItem(txt)
    except Exception as _e:
        print(f"跳过畸形标注: {_e}")
        continue
# ===== ai_engine.py (162 行) =====
# =====
# AI 推理引擎模块
# 负责：异步后台 AI 多器官分割推理 (organs.onnx, 25 类)
# 设计：纯 Python daemon 线程，避免继承 QThread 的析构崩溃风险；
# 完成/进度经 Qt 信号 (QueuedConnection) 投递回主线程更新 UI
# =====

from __future__ import annotations

import os
import threading
import time
from collections.abc import Callable

import numpy as np
from PySide6.QtCore import QObject, Signal

import segmentation
from constants import MODEL_PATH

class _AISignals(QObject):
    """跨线程回调载体。在主线程创建，子线程 emit 时 Qt 以 QueuedConnection 自动投递到
    主线程事件循环——这是从 threading.Thread 安全更新 Qt UI 的正确方式。
    (不能用 QTimer.singleShot: 它依附调用它的子线程，而子线程无 Qt 事件循环，回调不 fire。)"""
    finished = Signal(object, float) # (label_map, elapsed_ms)
    progress = Signal(int, int)      # (done_slices, total_slices)

try:
    import onnxruntime as ort
    HAS_ONNX = True
except ImportError:
    HAS_ONNX = False

# 模块级 InferenceSession 缓存：模型加载 (含 119MB 外部权重) 有固定开销，
# 按 model_path 缓存复用，避免每次推理 (尤其快速切换数据) 重复加载。
# onnxruntime 的 session.run 是线程安全的，可被多个后台线程共享。
_SESSION_CACHE = {}
```

```

def _get_session(model_path: str):
    sess = _SESSION_CACHE.get(model_path)
    if sess is None:
        so = ort.SessionOptions()
        so.enable_cpu_mem_arena = False # 关内存池, 显著降低分块推理的峰值内存
        sess = ort.InferenceSession(model_path, sess_options=so,
                                    providers=['CPUExecutionProvider'])
        _SESSION_CACHE[model_path] = sess
    return sess

class AutoAIEngineThread:
    """在后台线程中执行 AI 多器官分割推理, 完成后通过 Qt 信号安全回调主线程。"""

    def __init__(self, volume_hu: np.ndarray,
                 callback: Callable[[np.ndarray, float], None],
                 model_path: str = MODEL_PATH,
                 progress_callback: Callable[[int, int], None] | None = None) -> None:
        # volume_hu: 完整的 3D HU 值体素数组, shape=(Z, H, W), float32
        # callback: 推理完成后调用, 签名为 callback(label_map, elapsed_ms)
        # label_map 为 uint8 多类标签图 (0=背景, 1-24=器官类别)
        # model_path: ONNX 模型文件绝对路径, 默认 constants.MODEL_PATH (models/organs.onnx)
        # progress_callback: 可选, 签名 progress_callback(done_slices, total_slices),
        #                   在滑动推理过程中经 QTimer 投递到主线程, 用于更新进度显示
        self.volume_hu = volume_hu
        self.callback = callback
        self.model_path = model_path
        self.progress_callback = progress_callback
        self._thread = None
        # 信号对象在此 (主线程) 创建, 其槽即在主线程执行; 子线程 emit 自动排队投递
        self._signals = _AISignals()
        self._signals.finished.connect(lambda m, t: self.callback(m, t))
        if progress_callback is not None:
            self._signals.progress.connect(lambda d, t: self.progress_callback(d, t))
        # 单次推理约 8.8GB 内存、~100s 且不可中途中断 onnxruntime.run, 快速切换数据时
        # 若不作废旧线程, 多个推理会并发叠加导致内存翻倍甚至 OOM. cancel() 置位后,
        # 滑动循环在下一个 z 块边界提前退出并放弃回调, 及时释放内存。
        self._cancelled = False

    def start(self) -> None:
        # daemon=True 确保主窗口关闭后不会因后台线程阻塞进程退出
        self._thread = threading.Thread(target=self._run, daemon=True)
        self._thread.start()

    def cancel(self) -> None:
        """作废本次推理: 滑动循环会在下一个 z 块边界停止, 且不再触发完成回调。"""
        self._cancelled = True

```



```

def isRunning(self) -> bool:
    return self._thread is not None and self._thread.is_alive()

def _run(self) -> None:
    """推理主体，运行在后台线程，严禁在此处操作任何 Qt 对象（非线程安全）。"""
    start_t = time.perf_counter()

    # HU 值归一化到 [0, 1]: 肺窗范围 -1000~400 HU 覆盖空气到软组织
    # 超出范围的 HU 值 (骨骼 > 400) 通过 clip 截断，防止影响网络输入分布
    norm_vol = np.clip(self.volume_hu, -1000, 400)
    norm_vol = (norm_vol - (-1000)) / (400 - (-1000))
    norm_vol = norm_vol.astype(np.float32)

    final_mask = None

    # === 路径1: 真实 ONNX 多器官分割推理 ===
    if HAS_ONNX and os.path.exists(self.model_path):
        try:
            final_mask = self._run_onnx_multiorgan(norm_vol)
        except Exception as e:
            print(f"ONNX 推理失败，降级为数学算法: {e}")

    # 推理被作废 (用户已切换到新数据): 直接退出，不回调、不做数学降级，释放内存
    if self._cancelled:
        return

    # === 路径2: 纯数学算法降级 (无模型文件或推理失败时自动启用) ===
    # 逻辑已抽到 segmentation.segment_lungs_fallback (无 Qt, 可独立单测):
    # 阈值取低密度空气 -> 3D 连通域 -> 剔除体表边界相连的体外空气 -> 剩余内部空气取
    # 最大 (及其5%的次大) 连通域为双肺。
    if final_mask is None:
        final_mask = segmentation.segment_lungs_fallback(self.volume_hu)

    if self._cancelled:
        return # 数学降级期间也可能被作废，退出前再确认一次
    final_mask = final_mask.astype(np.uint8) # 统一为 uint8 标签图，供调色板 LUT 索引
    end_t = time.perf_counter()
    # 经信号跨线程投递到主线程 (QueuedConnection)，安全更新 Qt UI
    self.signals.finished.emit(final_mask, (end_t - start_t) * 1000)

def _run_onnx_multiorgan(self, norm_vol: np.ndarray) -> np.ndarray | None:
    """ONNX 多器官分割推理，返回 uint8 标签图 (0=背景, 1-24=器官类别)。

    关键约束 (均由对 organs.onnx 的实测确定):
    - 输入 (1,1,D,H,W)，每个空间维必须 pad 到 32 的倍数；
    - 必须整幅 xy 送入，做中心裁剪会破坏全局上下文导致器官被误判为背景；
    - 输出 (1,25,D,H,W) 为 logits，取 argmax(axis=1) 得类别，而非阈值二值化；
    - 整卷一次推理输出约 6.7GB 会 OOM，故沿 z 分块滑动 (DZ=32) + 关闭
      CPU 内存池，把峰值压到约 8.8GB。
    """

```

```

Z, H, W = norm_vol.shape
session = _get_session(self.model_path) # 复用缓存的会话, 避免重复加载模型
input_name = session.get_inputs()[0].name

ph, pw = (-H) % 32, (-W) % 32 # xy 方向对齐到 32 的倍数所需的填充
seg = np.zeros((Z, H, W), dtype=np.uint8)
DZ = 32
for z0 in range(0, Z, DZ):
    if self._cancelled:
        return None # 已作废: 停止推理, 让 _run 放弃回调并释放内存
    z1 = min(z0 + DZ, Z)
    blk = norm_vol[z0:z1]
    pd = (-blk.shape[0]) % 32 # z 方向 (尤其末块) 对齐到 32
    if pd or ph or pw:
        blk = np.pad(blk, ((0, pd), (0, ph), (0, pw)), mode='constant')
    out = session.run(None, {input_name: blk[np.newaxis, np.newaxis].astype(np.float32)})[0][0]
    lab = out.argmax(0).astype(np.uint8) # (D',H',W')
    seg[z0:z1] = lab[z1 - z0, :H, :W] # 裁掉 pad 部分写回原尺寸
    del out, lab
    if self.progress_callback is not None:
        # 经信号跨线程投递到主线程更新进度显示 (子线程禁止直接操作 Qt)
        self._signals.progress.emit(z1, Z)
    return seg
# ===== graphics_view.py (609 行) =====
# =====
# 医学影像自定义视图组件模块
# 负责: QGraphicsView 扩展, 实现影像交互 (缩放/平移/调窗/标注/MPR 十字线)
# =====

import math
import uuid

from PySide6.QtCore import QPointF, QPoint, QRectF, Qt, QTimer, Signal
from PySide6.QtGui import (
    QBrush,
    QColor,
    QFont,
    QMouseEvent,
    QPainter,
    QPainterPath,
    QPen,
    QPixmap,
    QPolygonF,
    QTransform,
)
from PySide6.QtWidgets import (
    QGraphicsEllipseItem,
    QGraphicsItem,
    QGraphicsLineItem,
    QGraphicsPathItem,

```

```

    QGraphicsPixmapItem,
    QGraphicsPolygonItem,
    QGraphicsRectItem,
    QGraphicsScene,
    QGraphicsTextItem,
    QGraphicsView,
)

# 常量集中定义在 constants.py, 此模块只需要工具 ID (鼠标事件分发用)
from constants import (
    TOOL_AI_TRACK,
    TOOL_CROP,
    TOOL_DRAW,
    TOOL_POINTER,
    TOOL_RECT_CROP,
    TOOL_ROI,
    TOOL_RULER,
    TOOL_SEG_BRUSH,
    TOOL_SEG_ERASE,
)

class ROIGraphicsItem(QGraphicsEllipseItem):
    """可拖动+可缩放的椭圆 ROI。
    - 拖内部: 整体移动; 拖右下角手柄: 缩放。
    - 交互结束后把新几何写回 annotation['rect'] (持久层), 并经回调请求主窗口重算统计+重绘。
    - 用 QGraphicsItem 自身的事件处理 (需视图处于 NoDrag 才能收到; 命中判定在视图层完成)。
    """
    HANDLE = 12 # 右下角缩放手柄边长 (像素)

    def __init__(self, anno, on_changed):
        x, y, w, h = anno['rect']
        super().__init__(0.0, 0.0, w, h) # rect 以 (0,0) 为基准, 位置用 setPos 表示
        self.setPos(x, y)
        self._anno = anno
        self._on_changed = on_changed
        self._resizing = False
        self.setFlag(QGraphicsItem.ItemIsMovable, True)
        self.setFlag(QGraphicsItem.ItemIsSelectable, True)
        # 右下角缩放手柄: 纯视觉子图元, 不接收鼠标 (命中由父项按角点位置判定)
        self._handle = QGraphicsRectItem(w - self.HANDLE, h - self.HANDLE, self.HANDLE, self.HANDLE, self)
        self._handle.setAcceptedMouseButtons(Qt.NoButton)

    def set_appearance(self, color):
        self.setPen(QPen(color, 2))
        self._handle.setPen(QPen(color, 1))
        self._handle.setBrush(QBrush(color))

    def _sync_handle(self):
        r = self.rect()

```

```

self._handle.setRect(r.width() - self.HANDLE, r.height() - self.HANDLE, self.HANDLE, self.HANDLE)

def _commit(self):
    """把当前几何写回 annotation, 并延迟触发重算 (避免在事件中销毁自身)。"""
    p, r = self.pos(), self.rect()
    self._anno['rect'] = (p.x(), p.y(), r.width(), r.height())
    if self._on_changed:
        QTimer.singleShot(0, self._on_changed)

def mousePressEvent(self, ev):
    r = self.rect()
    # 命中右下角手柄区域 -> 进入缩放; 否则交给基类做整体移动
    if ev.pos().x() >= r.width() - self.HANDLE and ev.pos().y() >= r.height() - self.HANDLE:
        self._resizing = True
        ev.accept()
    else:
        self._resizing = False
        super().mousePressEvent(ev)

def mouseMoveEvent(self, ev):
    if self._resizing:
        neww = max(self.HANDLE * 2, ev.pos().x())
        newh = max(self.HANDLE * 2, ev.pos().y())
        self.setRect(0.0, 0.0, neww, newh)
        self._sync_handle()
        ev.accept()
    else:
        super().mouseMoveEvent(ev)

def mouseReleaseEvent(self, ev):
    was_resizing = self._resizing
    self._resizing = False
    if not was_resizing:
        super().mouseReleaseEvent(ev) # 完成移动
    self._commit()

# =====
# 自定义医学影像视图组件
# 继承 QGraphicsView, 在标准图形视图框架基础上叠加:
# - 右键拖拽调节窗宽/窗位 (Windowing)
# - 鼠标中键平移 (Pan)
# - Ctrl+滚轮缩放 (Zoom)
# - 普通滚轮切片 (Scroll)
# - 标注工具: 测距卡尺、自由画笔、套索、矩形截取、AI追踪框
# - MPR 十字准线叠加
# =====
class MedicalGraphicsView(QGraphicsView):
    # --- 自定义信号, 所有业务逻辑解耦到 MedicalViewer 中处理 ---
    clicked_pos = Signal(QPoint) # 鼠标左键点击 (指针工具), 用于测量 HU 值

```

```

wheel_scrolled = Signal(int) # 滚轮滚动量 (非 Ctrl), 用于切换切片
annotation_added = Signal(dict) # 标注完成, 携带标注数据字典
crop_requested = Signal(list) # 截取/套索完成, 携带多边形顶点列表
track_requested = Signal(QRectF) # 3D 追踪框选完成, 携带矩形区域
annotation_deleted = Signal(str) # Delete 键删除选中标注, 携带标注 id
window_changed = Signal(int, int) # 右键拖拽调节窗宽窗位的增量 (dWw, dWl)
mouse_hovered = Signal(QPoint) # 鼠标移动, 携带场景坐标 (整数像素), 用于 MPR 联动十字线
seg_paint_requested = Signal(list, bool) # 分割手动修正: (轨迹点列表, 是否橡皮擦除)

def __init__(self, view_id):
    super().__init__()
    self.view_id = view_id # 视图编号 1~4, 用于日志和信号路由
    self.current_tool = TOOL_POINTER
    self.pixel_spacing = (1.0, 1.0) # (行间距mm, 列间距mm), 从 DICOM PixelSpacing 读取

    # --- 场景与图层堆叠 ---
    # Z轴: image_item(0) < mask_item(1) < 十字线(2) < 标注(默认0, 后续addItem时不设置)
    # mask_item 覆盖在影像上方, 使用半透明 RGBA 叠加显示 AI 分割蒙版
    self.scene = QGraphicsScene(self)
    self.setScene(self.scene)
    self.image_item = QGraphicsPixmapItem()
    self.scene.addItem(self.image_item)
    self.mask_item = QGraphicsPixmapItem()
    self.mask_item.setZValue(1) # 确保蒙版始终渲染在影像图层上方
    self.scene.addItem(self.mask_item)

    # --- MPR 十字准线 (橙色虚线) ---
    # 十字线初始位置 (0,0), 在 draw_crosshair() 中动态更新坐标
    self.vline = QGraphicsLineItem()
    self.hline = QGraphicsLineItem()
    pen_cross = QPen(QColor("#F39C12"), 1, Qt.DashLine) # 橙色虚线, 1px 不遮挡诊断信息
    self.vline.setPen(pen_cross); self.hline.setPen(pen_cross)
    self.vline.setZValue(2); self.hline.setZValue(2) # 最顶层, 始终可见
    self.scene.addItem(self.vline); self.scene.addItem(self.hline)
    self.vline.hide(); self.hline.hide() # 默认隐藏, MPR 联动开启后才显示

    # 分割画笔/橡皮的光标预览圈: 选中画笔工具时跟随鼠标显示笔刷范围
    self.brush_cursor = QGraphicsEllipseItem()
    self.brush_cursor.setZValue(3)
    self.brush_cursor.setVisible(False)
    self.scene.addItem(self.brush_cursor)

    # --- 渲染与交互设置 ---
    self.setRenderHint(QPainter.Antialiasing) # 标注线条抗锯齿
    self.setRenderHint(QPainter.SmoothPixmapTransform) # 影像双线性插值缩放 (临床模式默认开启)
    self.setDragMode(QGraphicsView.NoDrag) # 默认无拖拽, 中键时临时切换为 ScrollHandDrag
    self.setHorizontalScrollBarPolicy(Qt.ScrollBarAlwaysOff)
    self.setVerticalScrollBarPolicy(Qt.ScrollBarAlwaysOff)
    self.setStyleSheet("background-color: #000000; border: none;")
    self.setAlignment(Qt.AlignCenter)

```

```

self.setMouseTracking(True) # 必须开启, 否则鼠标未按下时不触发 mouseMoveEvent (影响 MPR 十字线)

# --- 绘图状态变量 ---
self.is_drawing = False      # 当前是否处于绘制/框选操作中
self.is_windowing = False    # 右键拖拽调窗是否激活
self.last_mouse_pos = None    # 上一帧鼠标位置, 用于计算右键拖拽的增量
self.temp_item = self.temp_rect_item = self.temp_text = self.start_pos = self.current_path = None
self.polygon_points = []     # 套索/多边形工具的中间顶点列表

self.user_zoomed = False     # 用户是否 Ctrl+滚轮手动缩放过; 决定 resize 是否重新适配
self.brush_radius = 6        # 分割修正画笔/橡皮的半径 (像素), 由主窗口 spin 同步

# --- DICOM 信息叠层 (PACS 风格四角文字 + 解剖方位字母) ---
self.show_overlay = True     # 叠加显隐开关
self.overlay_lines = {}      # {'tl': [...], 'tr': [...], 'bl': [...], 'br': [...]} 四角文字行
self.orient_labels = {}      # {'top', 'bottom', 'left', 'right'} 解剖方位字母

def set_overlay(self, corners, orient):
    """设置四角信息文字与方位字母并触发重绘。corners/orient 由主窗口按当前切片构建。"""
    self.overlay_lines = corners
    self.orient_labels = orient
    self.viewport().update()

def drawForeground(self, painter, rect):
    """在视口像素坐标绘制 DICOM 叠层—resetTransform 使其固定在四角/四边,
    不随影像缩放平移移动 (医学影像软件的标准 overlay 行为)。"""
    super().drawForeground(painter, rect)
    pm = self.image_item.pixmap()
    if not self.show_overlay or pm.isNull():
        return
    painter.save()
    painter.resetTransform() # 切换到视口像素坐标系
    W, H = self.viewport().width(), self.viewport().height()
    m = 8
    painter.setFont(QFont("Menlo", 9))
    fm = painter.fontMetrics(); lh = fm.height()
    painter.setPen(QColor(150, 220, 220, 220)) # 淡青, PACS 信息文字色
    for i, s in enumerate(self.overlay_lines.get('tl', [])):
        painter.drawText(m, m + lh * (i + 1), s)
    for i, s in enumerate(self.overlay_lines.get('tr', [])):
        painter.drawText(W - m - fm.horizontalAdvance(s), m + lh * (i + 1), s)
    bl = self.overlay_lines.get('bl', [])
    for i, s in enumerate(bl):
        painter.drawText(m, H - m - lh * (len(bl) - 1 - i), s)
    br = self.overlay_lines.get('br', [])
    for i, s in enumerate(br):
        painter.drawText(W - m - fm.horizontalAdvance(s), H - m - lh * (len(br) - 1 - i), s)
    # 解剖方位字母 (四边中点), 琥珀色加粗
    painter.setPen(QColor(255, 205, 90, 235))
    painter.setFont(QFont("Menlo", 11, QFont.Bold))

```

```

    ofm = painter.fontMetrics()
    o = self.orient_labels
    if o.get('top'):
        painter.drawText(W // 2 - ofm.horizontalAdvance(o['top']) // 2, m + ofm.ascent(), o['top'])
    if o.get('bottom'):
        painter.drawText(W // 2 - ofm.horizontalAdvance(o['bottom']) // 2, H - m, o['bottom'])
    if o.get('left'):
        painter.drawText(m, H // 2 + ofm.ascent() // 2, o['left'])
    if o.get('right'):
        painter.drawText(W - m - ofm.horizontalAdvance(o['right']), H // 2 + ofm.ascent() // 2, o['right'])
    painter.restore()

def set_image(self, pixmap, mask_qimg=None, pixel_spacing=(1.0, 1.0)):
    """更新当前视图的影像和蒙版。

    fitInView 守卫逻辑:
        仅当变换矩阵的 m11 (水平缩放因子) 接近 1.0 时才执行自动适配。
        m11 == 1.0 意味着视图处于"原始/未缩放"状态 (刚切换切片或初始化)。
        一旦用户通过 Ctrl+滚轮放大, m11 != 1.0, 后续切片更新不会重置缩放,
        保留医生的放大查看状态——这是医学影像工作站的基本用户体验要求。
    """
    self.image_item.setPixmap(pixmap)
    self.pixel_spacing = pixel_spacing
    rect = pixmap.rect()
    # sceneRect 必须随图像尺寸更新, 否则 fitInView 会基于旧尺寸计算缩放比,
    # 导致多平面 (冠状/矢状面) 分辨率不同时显示错位
    self.scene.setSceneRect(QRectF(rect))
    # 十字线覆盖整个图像范围, 坐标系原点在图像左上角
    self.vline.setLine(0, 0, 0, rect.height())
    self.hline.setLine(0, 0, rect.width(), 0)
    if mask_qimg:
        self.mask_item.setPixmap(from_image(mask_qimg))
        self.mask_item.show()
    else:
        self.mask_item.hide()
    if abs(pixel_spacing[0] - pixel_spacing[1]) > 1e-9:
        # 各向异性 (冠/矢状面: 层厚≠面内像素间距): 按物理尺寸做非均匀适配,
        # 使显示比例符合解剖。每次都重适配 (MPR 面不保留 Ctrl 缩放, 可接受)。
        self._apply_aniso_fit()
    else:
        # 各向同性 (横断面/重建/对比):
        #   - m11=1 (identity/未缩放) → 适配;
        #   - m11≠m22 (从各向异性面切回来的残留非均匀变换) → 强制重适配, 否则横断面会
        #       带着上一帧冠/矢状面的拉伸变换显示;
        #   - 均匀缩放 (m11=m22≠1, 用户 Ctrl+滚轮) → 保留, 不重置。
        t = self.transform()
        if abs(t.m11() - 1.0) < 1e-6 or abs(t.m11() - t.m22()) > 1e-9:
            self.fitInView(self.scene.sceneRect(), Qt.KeepAspectRatio)

def _apply_aniso_fit(self):

```

```

"""各向异性像素的非均匀适配: 让每个体素按 (列间距=水平, 行间距=垂直) 的真实物理
尺寸显示, 整体等比缩放填满视口。关键: 只改 View 变换, 不缩放图元、不重采样。
故 scene 坐标仍严格等于体素索引——mapToScene 自动求逆, hover/测量/十字线坐标受影响。"""
rect = self.scene.sceneRect()
vw, vh = self.viewport().width(), self.viewport().height()
sp0, sp1 = self.pixel_spacing          # (行/垂直间距, 列/水平间距)
aw, ah = rect.width() * sp1, rect.height() * sp0  # 物理宽、高
if aw <= 0 or ah <= 0 or vw <= 0 or vh <= 0:
    return
f = min(vw / aw, vh / ah) * 0.98      # 0.98 留少量边距
# 缩放矩阵: 水平 f*sp1, 垂直 f*sp0; 无旋转/错切
self.setTransform(QTransform(f * sp1, 0.0, 0.0, f * sp0, 0.0, 0.0))
self.centerOn(rect.center())

def draw_crosshair(self, x, y, show=True):
    """在场景坐标 (x, y) 处绘制 MPR 十字准线。
    坐标越界或 show=False 时自动隐藏, 防止十字线超出影像范围造成视觉干扰。
    """
    if show and self.image_item.pixmap():
        w, h = self.image_item.pixmap().width(), self.image_item.pixmap().height()
        if 0 <= x < w and 0 <= y < h:
            self.vline.setLine(x, 0, x, h)
            self.hline.setLine(0, y, w, y)
            self.vline.show(); self.hline.show()
            return
        self.vline.hide(); self.hline.hide()

def clear_annotations(self):
    """移除场景中的所有标注图元, 保留底层影像、蒙版和十字线图元。
    每次 update_display() 刷新影像后都需要调用, 防止标注重影 (旧标注叠加在新帧上)。
    """
    keep = (self.image_item, self.mask_item, self.vline, self.hline, self.brush_cursor)
    for item in self.scene.items():
        if item.parentItem() is not None:
            continue # 子图元 (如 ROI 缩放手柄) 随父项一起移除, 不单独处理
        if item not in keep:
            self.scene.removeItem(item)

def leaveEvent(self, event):
    """鼠标离开视图时隐藏画笔预览圈。"""
    self.brush_cursor.setVisible(False)
    super().leaveEvent(event)

def resizeEvent(self, event):
    """布局变化 (分割条拖动、窗口缩放) 触发自动重新适配影像到视图。
    这是 resizeEvent 的核心用途: 当父容器尺寸改变后, 视图需要重新计算
    fitInView 以填满新空间。仅在有效影像时触发, 避免对空视图操作。
    """
    super().resizeEvent(event)
    px = self.image_item.pixmap()

```



```

# 用户手动缩放过则保持其缩放, 不因 resize 强制回到适配 (与 set_image 的保留缩放一致)
if px and not px.isNull() and not self._user_zoomed:
    if abs(self.pixel_spacing[0] - self.pixel_spacing[1]) > 1e-9:
        self._apply_aniso_fit() # 各向异性面: 按物理比例重适配
    else:
        self.fitInView(self.scene.sceneRect(), Qt.KeepAspectRatio)

def wheelEvent(self, event):
    """滚轮事件分发:
    - Ctrl + 滚轮: 以鼠标位置为锚点缩放 (每档 x1.15 或 ÷1.15)
    - 普通滚轮: 发射 wheel_scrolled 信号, 由父窗口决定切换哪个方向的切片
    """
    if event.modifiers() == Qt.ControlModifier:
        # AnchorUnderMouse 使缩放中心固定在光标下方, 符合影像阅片习惯
        self.setTransformationAnchor(QGraphicsView.AnchorUnderMouse)
        z = 1.15 if event.angleDelta().y() > 0 else 1 / 1.15
        self.scale(z, z)
    else:
        self.wheel_scrolled.emit(event.angleDelta().y())

def mousePressEvent(self, event):
    """鼠标按下: 根据当前工具初始化对应的绘制状态, 或开始平移/调窗操作。"""
    # 中键按下: 将拖拽模式切换为 ScrollHandDrag (手型游标), 实现图像平移
    # 技巧: Qt 的 ScrollHandDrag 需要左键触发, 这里构造一个假的左键事件欺骗基类
    if event.button() == Qt.MidButton:
        self.setDragMode(QGraphicsView.ScrollHandDrag)
        fake = QMouseEvent(event.type(), event.position(), event.globalPosition(),
                           Qt.LeftButton, Qt.LeftButton, event.modifiers())
        super().mousePressEvent(fake)
        return

    # 右键按下: 进入窗宽/窗位拖拽调节模式 (放射科标准交互: 水平拖 → 调 WW, 垂直拖 → 调 WL)
    if event.button() == Qt.RightButton:
        self.is_windowing = True
        self.last_mouse_pos = event.position().toPoint()
        self.setCursor(Qt.SizeAllCursor) # 四向箭头光标提示用户可以拖拽
        return

    # 将视图坐标 (像素点) 转换为场景坐标 (影像素坐标), 供各工具使用
    sp = self.mapToScene(event.position().toPoint())

    if event.button() == Qt.LeftButton:
        self.is_drawing = True

        if self.current_tool == TOOL_POINTER:
            # 先命中测试: 点在 ROI (或其缩放手柄) 上 → 交给该 item 自行移动/缩放,
            # 视图设 NoDrag (否则 ScrollHandDrag 会拦截拖拽去平移视图), 且不测 HU
            hit = self.itemAt(event.position().toPoint())
            if isinstance(hit, ROIGraphicsItem) or (hit is not None and isinstance(hit.parentItem(), ROIGraphicsItem)):

```

```
        self.setDragMode(QGraphicsView.NoDrag)
        super().mousePressEvent(event)
        return
    # 指针工具: 点击发射 HU 测量信号, 同时允许拖拽平移 (ScrollHandDrag)
    self.clicked_pos.emit(event.position().toPoint())
    self.setDragMode(QGraphicsView.ScrollHandDrag)
    super().mousePressEvent(event)

elif self.current_tool == TOOL_RULER:
    # 卡尺工具: 记录起点, 创建临时直线和距离文字标签, 在 Move 中实时更新
    self.start_pos = sp
    pen = QPen(QColor("#FF3366"), 2)
    self.temp_item = QGraphicsLineItem(QLineF(sp, sp))
    self.temp_item.setPen(pen)
    self.scene.addItem(self.temp_item)
    self.temp_text = QGraphicsTextItem("")
    self.temp_text.setDefaultTextColor(QColor("#FF3366"))
    self.temp_text.setFont(QFont("Arial", 11, QFont.Bold))
    self.scene.addItem(self.temp_text)

elif self.current_tool == TOOL_DRAW:
    # 自由画笔: 用 QPainterPath 记录连续轨迹, RoundCap/RoundJoin 让线条圆润
    self.current_path = QPainterPath(sp)
    self.polygon_points = [(sp.x(), sp.y())]
    self.temp_item = QGraphicsPathItem(self.current_path)
    pen = QPen(QColor("#00ADB5"), 2)
    pen.setCapStyle(Qt.RoundCap)
    pen.setJoinStyle(Qt.RoundJoin)
    self.temp_item.setPen(pen)
    self.scene.addItem(self.temp_item)

elif self.current_tool == TOOL_CROP:
    # 套索工具: 连续记录顶点, 形成任意多边形, 半透明黄色填充预览
    self.polygon_points = [sp]
    self.temp_item = QGraphicsPolygonItem(QPolygonF(self.polygon_points))
    self.temp_item.setPen(QPen(QColor("#F1C40F"), 2, Qt.DashLine))
    self.temp_item.setBrush(QBrush(QColor(241, 196, 15, 50))) # alpha=50 半透明
    self.scene.addItem(self.temp_item)

elif self.current_tool in [TOOL_RECT_CROP, TOOL_AI_TRACK]:
    # 矩形截取/AI追踪: 橙色矩形 vs 紫色矩形, 视觉区分不同功能
    self.start_pos = sp
    c = QColor("#9B59B6") if self.current_tool == TOOL_AI_TRACK else QColor("#E67E22")
    self.temp_rect_item = QGraphicsRectItem(QRectF(sp, sp))
    self.temp_rect_item.setPen(QPen(c, 2, Qt.DashLine))
    self.temp_rect_item.setBrush(QBrush(QColor(c.red(), c.green(), c.blue(), 40)))
    self.scene.addItem(self.temp_rect_item)

elif self.current_tool in [TOOL_SEG_BRUSH, TOOL_SEG_ERASE]:
    # 分割修正: 画笔(绿)补画 / 橡皮(红)擦除, 拖拽实时预览, release 时提交到蒙版
```

```

        self.current_path = QPainterPath(sp)
        self.polygon_points = [(sp.x(), sp.y())]
        self.temp_item = QGraphicsPathItem(self.current_path)
        is_erase = self.current_tool == TOOL_SEG_ERASE
        c = QColor(231, 76, 60, 150) if is_erase else QColor(46, 204, 113, 150)
        pen = QPen(c, self.brush_radius * 2) # 笔宽=直径, 预览与实际写入范围一致
        pen.setCapStyle(Qt.RoundCap); pen.setJoinStyle(Qt.RoundJoin)
        self.temp_item.setPen(pen)
        self.scene.addItem(self.temp_item)

    elif self.current_tool == TOOL_ROI:
        # 椭圆 ROI: 拖出外接矩形, 绘制内切椭圆, release 时提交为 roi 标注
        self.start_pos = sp
        self.temp_rect_item = QGraphicsEllipseItem(QRectF(sp, sp))
        self.temp_rect_item.setPen(QPen(QColor("#F1C40F"), 2))
        self.temp_rect_item.setBrush(QBrush(QColor(241, 196, 15, 30)))
        self.scene.addItem(self.temp_rect_item)

def mouseMoveEvent(self, event):
    """鼠标移动: 实时更新绘制预览 / 调窗增量 / MPR 十字线位置。"""
    sp = self.mapToScene(event.position().toPoint())

    # 分割工具: 光标预览圈跟随鼠标显示笔刷范围 (画笔绿/橡皮红)
    if self.current_tool in [TOOL_SEG_BRUSH, TOOL_SEG_ERASE]:
        r = self.brush_radius
        self.brush_cursor.setRect(sp.x() - r, sp.y() - r, 2 * r, 2 * r)
        erase = self.current_tool == TOOL_SEG_ERASE
        self.brush_cursor.setPen(QPen(QColor(231, 76, 60, 180) if erase else QColor(46, 204, 113, 180), 1, Qt.DashLine))
        self.brush_cursor.setVisible(True)
    elif self.brush_cursor.isVisible():
        self.brush_cursor.setVisible(False)

    # 非调窗状态下, 发射鼠标悬浮信号供 MPR 联动十字线更新
    # 调窗期间不更新十字线, 避免光标移动引起视图混乱
    if not self.is_windowing:
        real_coord = self.get_real_coordinates(event.position().toPoint())
        if real_coord:
            self.mouse_hovered.emit(QPoint(real_coord[0], real_coord[1]))

    if self.is_windowing:
        if self.last_mouse_pos is not None:
            curr_pos = event.position().toPoint()
            dx = curr_pos.x() - self.last_mouse_pos.x()
            dy = curr_pos.y() - self.last_mouse_pos.y()
            # x2 放大灵敏度: 原始像素增量较小, 乘以系数后 WW/WL 变化更明显
            self.window_changed.emit(dx * 2, dy * 2)
            self.last_mouse_pos = event.position().toPoint()
        return

    if self.is_drawing:

```

```

        if self.current_tool == TOOL_RULER and self.temp_item:
            # 实时更新直线终点, 并换算为毫米距离显示
            # pixel_spacing[0]=行间距(mm/像素), [1]=列间距, 分别对应 Y 轴和 X 轴
            self.temp_item.setLine(QLineF(self.start_pos, sp))
            d = math.sqrt(
                ((sp.x() - self.start_pos.x()) * self.pixel_spacing[1]) ** 2 +
                ((sp.y() - self.start_pos.y()) * self.pixel_spacing[0]) ** 2
            )
            self.temp_text.setPlainText(f"{d:.1f} mm")
            self.temp_text.setPos(sp.x() + 10, sp.y() + 10) # 标签跟随终点偏移显示

        elif self.current_tool == TOOL_DRAW and self.temp_item:
            # 追加路径节点, 实时渲染自由曲线
            self.current_path.lineTo(sp)
            self.temp_item.setPath(self.current_path)
            self.polygon_points.append((sp.x(), sp.y()))

        elif self.current_tool == TOOL_CROP and self.temp_item:
            # 套索: 每个 Move 事件都追加顶点, 形成连续多边形
            self.polygon_points.append(sp)
            self.temp_item.setPolygon(QPolygonF(self.polygon_points))

        elif self.current_tool in [TOOL_RECT_CROP, TOOL_AI_TRACK, TOOL_ROI] and self.temp_rect_item:
            # normalized() 保证无论拖拽方向如何, 矩形 left < right, top < bottom
            # QGraphicsEllipseItem 与 RectItem 同样支持 setRect (ROI 复用此逻辑)
            self.temp_rect_item.setRect(QRectF(self.start_pos, sp).normalized())

        elif self.current_tool in [TOOL_SEG_BRUSH, TOOL_SEG_ERASE] and self.temp_item:
            # 分割修正: 追加轨迹点, 实时更新画笔/橡皮预览路径
            self.current_path.lineTo(sp)
            self.temp_item.setPath(self.current_path)
            self.polygon_points.append((sp.x(), sp.y()))

    super().mouseMoveEvent(event)

def mouseReleaseEvent(self, event):
    """鼠标释放: 结束绘制操作, 提交标注数据或触发相应信号。"""
    # 中键释放: 恢复无拖拽模式
    if event.button() == Qt.MidButton:
        self.setDragMode(QGraphicsView.NoDrag)
        super().mouseReleaseEvent(event)
        return
    # 右键释放: 退出调窗模式, 光标恢复箭头
    if event.button() == Qt.RightButton:
        self.is_windowing = False
        self.last_mouse_pos = None
        self.setCursor(Qt.ArrowCursor)
        return

    if event.button() == Qt.LeftButton and self.is_drawing:

```

```
self.is_drawing = False

if self.current_tool == TOOL_POINTER:
    self.setDragMode(QGraphicsView.NoDrag) # 指针松开后恢复无拖拽

elif self.current_tool == TOOL_RULER and self.temp_item:
    # 移除临时预览图元, 将最终数据打包为字典发射信号
    # 由 MedicalViewer.handle_annotation_added 接收并持久化
    p2 = self.mapToScene(event.position().toPoint())
    d = {'id': str(uuid.uuid4()), 'type': 'ruler',
        'p1': (self.start_pos.x(), self.start_pos.y()),
        'p2': (p2.x(), p2.y())}
    self.scene.removeItem(self.temp_item)
    self.scene.removeItem(self.temp_text)
    self.annotation_added.emit(d)

elif self.current_tool == TOOL_DRAW and self.temp_item:
    d = {'id': str(uuid.uuid4()), 'type': 'path', 'points': self.polygon_points}
    self.scene.removeItem(self.temp_item)
    self.annotation_added.emit(d)

elif self.current_tool == TOOL_CROP and self.temp_item:
    pts = [(p.x(), p.y()) for p in self.polygon_points]
    self.scene.removeItem(self.temp_item)
    if len(pts) >= 3: # 至少三点才能构成多边形, 过滤误触
        self.crop_requested.emit(pts)

elif self.current_tool == TOOL_RECT_CROP and getattr(self, 'temp_rect_item', None):
    r = self.temp_rect_item.rect()
    self.scene.removeItem(self.temp_rect_item)
    if r.width() > 5: # 过滤宽度过小的误点 (< 5 像素视为无效框选)
        self.crop_requested.emit([
            (r.left(), r.top()), (r.right(), r.top()),
            (r.right(), r.bottom()), (r.left(), r.bottom())
        ])

elif self.current_tool == TOOL_AI_TRACK and getattr(self, 'temp_rect_item', None):
    r = self.temp_rect_item.rect()
    self.scene.removeItem(self.temp_rect_item)
    if r.width() > 5:
        self.track_requested.emit(r) # 发射 QRectF, 供 3D 连通域追踪使用

elif self.current_tool in [TOOL_SEG_BRUSH, TOOL_SEG_ERASE] and self.temp_item:
    # 分割修正: 移除预览, 把轨迹点交给主窗口写入 volume_mask
    self.scene.removeItem(self.temp_item)
    self.seg_paint_requested.emit(self.polygon_points, self.current_tool == TOOL_SEG_ERASE)

elif self.current_tool == TOOL_ROI and getattr(self, 'temp_rect_item', None):
    # 椭圆 ROI: 提交为 roi 标注, 统计在主窗口渲染时从 volume_hu 计算
    r = self.temp_rect_item.rect()
```

```

        self.scene.removeItem(self.temp_rect_item)
        if r.width() > 3 and r.height() > 3:
            self.annotation_added.emit({'id': str(uuid.uuid4()), 'type': 'roi',
                                       'rect': (r.x(), r.y(), r.width(), r.height())})

        # 清理临时图元引用, 防止悬空指针
        self.temp_item = self.temp_rect_item = self.temp_text = None

    super().mouseReleaseEvent(event)

    def keyPressEvent(self, event):
        """Delete/Backspace 键删除当前选中的标注图元 (需图元设置了 tooltip 作为 ID)。"""
        if event.key() in (Qt.Key_Delete, Qt.Key_Backspace):
            for item in self.scene.selectedItems():
                # tooltip 存储标注的 UUID, 通过信号通知父窗口从数据层删除
                if item.tooltip():
                    self.annotation_deleted.emit(item.tooltip())
            super().keyPressEvent(event)

    def get_real_coordinates(self, pos):
        """将视图坐标转换为影像像素坐标, 越界则返回 None。"""
        sp = self.mapToScene(pos)
        x, y = int(sp.x()), int(sp.y())
        if (self.image_item.pixmap() and
            0 <= x < self.image_item.pixmap().width() and
            0 <= y < self.image_item.pixmap().height()):
            return x, y
        return None

    def from_image(qimg):
        """QImage -> QPixmap 辅助函数, 集中处理 fromImage 调用。"""
        return QPixmap.fromImage(qimg)
# ===== recon.py (485 行) =====
# =====
# 重建算法纯计算模块
# 负责: 所有 CT 重建相关的纯数值计算, 不依赖任何 Qt/UI 代码
# 调用方: MedicalViewer 的方法作为薄包装层, 负责读取 UI 状态并展示结果
# =====

from __future__ import annotations

import hashlib
import inspect
import multiprocessing as _mp
import os
import time
from collections.abc import Callable

import numpy as np

```

```
import scipy.ndimage as ndimage
from scipy.interpolate import LinearNDInterpolator
from scipy.spatial import Delaunay
from skimage.transform import iradon, radon

# -----
# 模块级缓存（顶部集中声明，便于阅读时一眼看清全局可变状态）
# -----

# 系统矩阵磁盘缓存目录：放在本模块所在目录下的 .matrix_cache/,
# 避免与项目根混淆；首次访问时按需创建。
_MATRIX_CACHE_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), ".matrix_cache")

# 圆形掩码缓存：{n: mask}。同 n 在多次重建中反复使用，避免重复 ogrid + 比较运算。
# key 空间极小 (≤ 4 种 n)，无需淘汰策略。
_CIRCLE_MASK_CACHE = {}

# DFR Delaunay 三角剖分缓存：key=(num_detectors, len(theta), θ_start, θ_end)。
# scipy.griddata 内部每次都会重做 Delaunay 三角剖分 (O(N log N) 但常数极大)，
# 同参数复算时复用 Delaunay 对象可省去 60%~80% 的 DFR 总耗时：
# 数学上完全等价，三角剖分由几何点集唯一确定。
# key 空间小 (≤ 4 探测器数 × 4 角度配置 ≈ 16 entry)，无需淘汰策略。
_DFR_TRI_CACHE = {}

# -----
# 系统矩阵并行 worker（必须是模块顶层函数，multiprocessing 才能 pickle）
# -----

def _matrix_worker(args: tuple) -> tuple[int, int, np.ndarray]:
    """计算系统矩阵 A 中 [start_j, end_j) 列对应像素的 Radon 贡献。
    子进程独立导入 skimage，避免父进程 Qt 状态被复制到子进程。
    """
    start_j, end_j, n, theta = args
    import numpy as _np
    from skimage.transform import radon as _radon

    # 先跑一次空图确定探测器数量
    n_rays = _radon(_np.zeros((n, n), dtype=_np.float32), theta=theta, circle=True).size
    cols = _np.zeros((n_rays, end_j - start_j), dtype=_np.float32)
    img = _np.zeros((n, n), dtype=_np.float32)
    for k, j in enumerate(range(start_j, end_j)):
        r, c = j // n, j % n
        img[r, c] = 1.0
        cols[:, k] = _radon(img, theta=theta, circle=True).ravel()
        img[r, c] = 0.0
    return start_j, end_j, cols

# _matrix_worker 源码的 SHA1 前 8 位—嵌入缓存文件名中，
```

```

# worker 代码一改哈希就变, 旧缓存自动失效, 永远不需要手动清理 .matrix_cache/.
_WORKER_HASH = hashlib.sha1(
    inspect.getsource(_matrix_worker).encode('utf-8')
).hexdigest()[:8]

def _purge_stale_matrix_cache() -> None:
    """启动时清理 .matrix_cache/ 中哈希与当前 _WORKER_HASH 不匹配的过期文件。

    文件名规范: A_n{n}_na{na}_t{ts}_{te}_{hash8}.npz, 其中 hash8 必须是 8 位十六进制;
    任何不严格匹配此模式的文件一律跳过, 杜绝误删风险。
    """
    if not os.path.isdir(_MATRIX_CACHE_DIR):
        return
    hex_chars = set("0123456789abcdef")
    for fn in os.listdir(_MATRIX_CACHE_DIR):
        if not fn.startswith("A_n") or not fn.endswith(".npz"):
            continue
        stem = fn[:-4] # 去掉 .npz
        parts = stem.rsplit("_", 1)
        if len(parts) != 2:
            continue
        hash_part = parts[1]
        # 严格校验: hash 段必须正好 8 位且全部是小写十六进制字符
        if len(hash_part) != 8 or not all(c in hex_chars for c in hash_part):
            continue
        if hash_part == _WORKER_HASH:
            continue
        try:
            os.remove(os.path.join(_MATRIX_CACHE_DIR, fn))
        except OSError as e:
            print(f"Warning: failed to remove stale cache {fn}: {e}")

_purge_stale_matrix_cache()

# -----
# 正向投影
# -----

def compute_sinogram(img_norm: np.ndarray, theta: np.ndarray) -> np.ndarray:
    """对归一化图像执行 Radon 变换, 返回弦图。

    img_norm: 归一化到 [0,1] 的 2D 浮点数组
    theta: 投影角度数组 (度), 如 np.linspace(0, 180, 180, endpoint=False)
    返回: sinogram, shape=(探测器数, 角度数)
    """
    # circle=True: 只处理图像内切圆区域, 与 iradon 默认行为一致, 避免角落 padding 引入伪影
    return radon(img_norm, theta=theta, circle=True)

```



```

def make_theta(angle_range: float, n_proj: int | None = None) -> np.ndarray:
    """在 [0, angle_range) 度范围内均匀生成 n_proj 个投影角度。

    angle_range: 角度覆盖范围 (度)。180 为 CT 完整半圈 (对径投影冗余, 覆盖完整)。
    n_proj:      投影数量。None 时取 angle_range (每度一个投影, 向后兼容旧行为)。
                n_proj > angle_range 即"过采样"——角度间隔更细 (如 180°取360个=0.5°间隔),
                弦图信息更充分, 反投影/迭代重建质量更高 (代价是耗时随投影数线性增加)。
    返回:       np.ndarray, 长度 = n_proj, 范围 [0, angle_range)
    """
    if n_proj is None:
        n_proj = angle_range
    return np.linspace(0., float(angle_range), int(n_proj), endpoint=False)

# -----
# BP / FBP
# -----

def compute_bp(sinogram: np.ndarray, theta: np.ndarray) -> np.ndarray:
    """纯反投影 (不滤波), 返回重建图像。

    缺陷: 低频分量被过度叠加, 边缘极度模糊 (星形伪影)。
    对比目的: 展示滤波 (FBP) 对图像质量的改善效果。
    """
    # filter_name=None 表示纯反投影, 不做任何频域滤波
    return iradon(sinogram, theta=theta, filter_name=None, circle=True)

def compute_fbp(sinogram: np.ndarray, theta: np.ndarray, filter_name: str) -> tuple[np.ndarray, np.ndarray]:
    """滤波反投影 (FBP), 返回 (recon_bp_unfiltered, recon_fbp)。

    同时返回未滤波结果供对比展示, 避免调用方重复计算。
    filter_name: 'ramp' / 'shepp-logan' / 'cosine' / 'hamming' / 'hann'
                注意: UI 显示 'Ram-Lak', 调用前需映射为 'ramp'
    """
    # skimage.transform.iradon 内部滤波器名为 'ramp', 调用前需做名称映射
    if filter_name.lower() == "ram-lak":
        filter_name = "ramp"
    recon_bp = iradon(sinogram, theta=theta, filter_name=None, circle=True)
    recon_fbp = iradon(sinogram, theta=theta, filter_name=filter_name, circle=True)
    return recon_bp, recon_fbp

# -----
# DFR (直接傅里叶重建)
# -----

def compute_dfr(sinogram: np.ndarray, theta: np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray]:

```

```

"""直接傅里叶重建法 (DFR)，基于傅里叶中心切片定理。

返回: (freq_domain_2d, fft_1d_display, recon_dfr)
freq_domain_2d: 插值后的二维复数频域矩阵, 供"二维频域分布"视图展示
fft_1d_display: log1p 压缩后的一维频谱幅度图, 供"一维FFT谱"视图展示
recon_dfr: 2D 逆 FFT 重建结果 (复数, 取 abs 后显示)

算法步骤:
1. 对弦图每列 (沿探测器方向) 做 1D FFT → 极坐标频域样本
2. 将极坐标 (r, θ) 样本插值到直角坐标网格 (griddata, method='linear')
3. 对插值后的 2D 频域做 2D 逆 FFT → 重建图像
"""
num_detectors, num_angles = sinogram.shape

# 步骤1: 对弦图沿探测器方向 (axis=0) 做 1D FFT
# ifftshift 将数据中心移到 FFT 起点 (左端), fft 计算, 再 fftshift 将零频移回中心
# 这样 proj_fft[num_detectors//2, :] 对应零频 (直流分量)
proj_fft = np.fft.fftshift(
    np.fft.fft(np.fft.ifftshift(sinogram, axes=0), axis=0),
    axes=0
)

# 提取供展示的 1D 频谱 (对数压缩后的幅度谱, log1p 避免 log(0) 的问题)
fft_1d_display = np.log1p(np.abs(proj_fft))

# 步骤2: 构建极坐标网格 (r 为频率半径, theta 为投影角度)
r = np.arange(num_detectors) - num_detectors // 2
r_grid, theta_grid = np.meshgrid(r, np.deg2rad(theta), indexing='ij')

# 极坐标 → 直角坐标转换: (r, θ) → (r·cosθ, r·sinθ) = (kx, ky)
x_polar = r_grid * np.cos(theta_grid)
y_polar = r_grid * np.sin(theta_grid)
points = np.column_stack((x_polar.flatten(), y_polar.flatten()))
values = proj_fft.flatten() # 对应每个极坐标点的复数频域值

# 目标直角网格 (与频域图像一一对应的均匀网格)
grid_x, grid_y = np.meshgrid(r, r, indexing='ij')

# 散点插值: 将不均匀的极坐标样本插值到均匀的直角坐标网格
# 性能优化: Delaunay 三角剖分对相同 (num_detectors, theta) 是确定的,
# 缓存复用避免每次 griddata 重做剖分 (DFR 主要瓶颈)。
# 数学完全等价: LinearNDInterpolator(tri, values) 与
# griddata(points, values, ..., method='linear') 用相同三角剖分 + 重心插值算法。
tri_key = (num_detectors, len(theta),
           round(float(theta[0]), 4), round(float(theta[-1]), 4))
tri = _DFR_TRI_CACHE.get(tri_key)
if tri is None:
    tri = Delaunay(points)
    _DFR_TRI_CACHE[tri_key] = tri
# fill_value=0: 超出极坐标覆盖范围的格点填零 (高频端无测量数据)

```

```

interp = LinearNDInterpolator(tri, values, fill_value=0)
freq_domain_2d = interp(grid_x, grid_y)

# 步骤3: 2D 逆 FFT 还原图像
# nan_to_num: griddata 在极端条件下可能产生 NaN/Inf, 需在 ifft2 前清零,
# 否则 NaN 会通过 FFT 线性运算扩散到整个重建图像
freq_domain_2d = np.nan_to_num(freq_domain_2d, nan=0.0, posinf=0.0, neginf=0.0)
# ifftshift 先将零频移回左上角 (FFT 约定原点位置), ifft2 计算, 再 fftshift 将图像中心化
recon_dfr = np.fft.fftshift(np.fft.ifft2(np.fft.ifftshift(freq_domain_2d)))

return freq_domain_2d, fft_1d_display, recon_dfr

# -----
# 辅助: 小图准备 / 上采样
# -----

def _finite_clip(arr: np.ndarray, n: int) -> np.ndarray:
    """把重建结果整理为可显示的有限 [0,1] 图: 先中和 NaN/±Inf, 再 clip 到 [0,1], reshape 成 n×n。
    对齐 DFR 已有做法 (compute_dfr 里对频域先 nan_to_num)。DMR/ART/SIRT 若遇病态/损坏输入
    (如弦图混入非有限值) 会产出 NaN, 而 np.clip 对 NaN 无效—NaN 会静默变黑图并污染 RMSE。
    此处保证任何情况下返回的都是确定、有限、可显示的图像。"""
    a = np.nan_to_num(arr.reshape(n, n), nan=0.0, posinf=1.0, neginf=0.0)
    return np.clip(a, 0.0, 1.0).astype(np.float32)

def _circle_mask(n: int) -> np.ndarray:
    """返回 n×n 的圆形掩码 (内切圆内为 1, 圆外为 0), float32。同 n 直接复用缓存。"""
    m = _CIRCLE_MASK_CACHE.get(n)
    if m is None:
        cy, cx = n // 2, n // 2
        Y, X = np.ogrid[:n, :n]
        m = ((Y - cy) ** 2 + (X - cx) ** 2 <= (n // 2) ** 2).astype(np.float32)
        _CIRCLE_MASK_CACHE[n] = m
    return m

def prepare_small_image(img_norm: np.ndarray, n: int, angle_range: float,
                        n_proj: int | None = None) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """将归一化图像缩小为 n×n, 施加圆形掩码后执行 Radon 变换。

    img_norm: 归一化到 [0,1] 的 2D 浮点数组 (原始切片)
    n: 目标边长 (16 / 32 / 64)
    angle_range: 角度覆盖范围 (度, 60 / 120 / 180 / 360)
    n_proj: 投影数量; None 时等于 angle_range。见 make_theta 的过采样说明。

    返回: (img_small, sinogram, theta)
    img_small: 缩小后加了圆形掩码的图像
    sinogram: 对 img_small 的 Radon 变换结果
    theta: 对应角度数组

```

```

    圆形掩码说明:
    radon(circle=True) 只处理内切圆区域; 若不施加此掩码,
    原图角落有值而重建角落为0, 误差图会在角落出现虚假大误差,
    迷惑用户误判算法质量。
    """
    h0, w0 = img_norm.shape
    # ndimage.zoom 使用样条插值缩放, clip 防止插值超出 [0,1] 范围 (Gibbs 现象)
    img_small = np.clip(
        ndimage.zoom(img_norm, (n / h0, n / w0)), 0.0, 1.0
    ).astype(np.float32)

    # 圆形掩码: 与 radon(circle=True) 的处理区域完全对齐 (缓存复用, 同 n 不重复计算)
    img_small = img_small * _circle_mask(n)

    theta = make_theta(angle_range, n_proj)
    sinogram = radon(img_small, theta=theta, circle=True)
    return img_small, sinogram, theta

def upscale_recon(arr: np.ndarray, n: int) -> np.ndarray:
    """用 np.kron 做最近邻整数倍上采样, 将小图放大到至少 256×256 显示。

    选择 kron 而非 zoom/resize 的原因:
    kron 执行严格的像素复制 (每个原像素变成 scale×scale 的色块),
    保留像素块感——教学中展示重建分辨率差异的重要视觉线索。
    双线性插值会"美化"粗糙的 16×16 重建结果, 失去教学价值。
    """
    scale = max(1, 256 // n)
    if scale == 1:
        return arr
    # np.kron(A, B): 将 A 的每个元素替换为 A[i,j]*B, 等价于像素块复制
    return np.kron(arr, np.ones((scale, scale), dtype=np.float32))

# -----
# 系统矩阵构建
# -----

def build_system_matrix(n: int, theta: np.ndarray, cached_A: np.ndarray | None,
                        cached_A_key: tuple | None,
                        progress_cb: Callable[[int, int], None] | None = None
                        ) -> tuple[np.ndarray, tuple]:
    """逐像素构建系统矩阵 A, 用于 DMR 和 ART/SIRT。

    n:          图像边长 (A 的列数 = n²)
    theta:      投影角度数组
    cached_A:  上次缓存的矩阵 (None 表示无缓存)
    cached_A_key: 上次缓存的 key
    progress_cb: 可选进度回调 progress_cb(j, n_pixels), 每步调用
    """

```

```

返回: (A, key)
A: 系统矩阵, shape=(n_rays, n²), float32
key: 本次计算的缓存键

缓存键 = (n, 角度数, 起始角, 终止角); 图像尺寸和角度配置不变时直接复用。
64×64 × 180角的 A 矩阵约需数分钟构建, 缓存节省大量等待时间。
"""
key = (n, len(theta), round(float(theta[0]), 4), round(float(theta[-1]), 4))
if cached_A is not None and cached_A_key == key:
    return cached_A, key

# 磁盘缓存命中: A 矩阵在 (n, n_angles, θ_start, θ_end) 完全相同时是确定值,
# 第一次算完后写盘, 之后所有进程启动都可秒级 np.load 复用, 无任何精度损失。
# 文件名嵌入 _WORKER_HASH: worker 代码改动后哈希变, 旧缓存被自然忽略 (无需手动清理)。
cache_file = os.path.join(
    _MATRIX_CACHE_DIR,
    f"A_{key[0]}_n_{key[1]}_t_{key[2]:.4f}_{key[3]:.4f}_{_WORKER_HASH}.npy"
)
if os.path.exists(cache_file):
    try:
        A = np.load(cache_file)
        return A, key
    except Exception as e:
        print(f"Warning: matrix cache {cache_file} corrupted, rebuilding: {e}")

n_pixels = n * n
n_rays = radon(np.zeros((n, n), dtype=np.float32), theta=theta, circle=True).size
A = np.zeros((n_rays, n_pixels), dtype=np.float32)

# 每个批次处理的像素数: 让每个 worker 大约承担 1/4 的工作量,
# 多批次 (batch 数 > worker 数) 使负载均衡更好
# os.cpu_count() 在无法探测时返回 None (回退 4); 不用 _mp.cpu_count() 后者探测不到会抛
# NotImplementedError。与项目其余处 (ai_engine/graphics_view/_read_dicom_dir) 保持同一写法。
n_workers = min(os.cpu_count() or 4, 8)
batch = max(32, n_pixels // (n_workers * 4))
jobs = [(i, min(i + batch, n_pixels), n, theta)
        for i in range(0, n_pixels, batch)]

completed = 0
# 'spawn' 避免 fork 复制 Qt 父进程状态到子进程导致崩溃
ctx = _mp.get_context('spawn')
with ctx.Pool(processes=n_workers) as pool:
    # imap_unordered: 哪个 batch 先算完先回来, 不等最慢的那个
    for start_j, end_j, cols in pool.imap_unordered(_matrix_worker, jobs):
        A[:, start_j:end_j] = cols
        completed += end_j - start_j
        if progress_cb is not None:
            progress_cb(completed, n_pixels)

```

```

# 写入磁盘缓存, 下次同参数直接 np.load 跳过整个并行构建过程
try:
    os.makedirs(_MATRIX_CACHE_DIR, exist_ok=True)
    np.save(cache_file, A)
except Exception as e:
    print(f"Warning: failed to write matrix cache {cache_file}: {e}")

return A, key

# -----
# DMR (直接矩阵重建)
# -----

def compute_dmr(A: np.ndarray, p_vec: np.ndarray, n: int) -> tuple[np.ndarray, float]:
    """用最小二乘法求解  $A \cdot x = p$ , 返回 (img_recon, error_time_ms)。

    A:      系统矩阵, shape=(n_rays, n2)
    p_vec:  弦图展平向量, shape=(n_rays,)
    n:      图像边长

    返回: (img_recon, elapsed_ms)
    img_recon: 重建图像, shape=(n, n), 值已 clip 到 [0, 1]
    elapsed_ms: lstsq 求解耗时 (毫秒)

    数学原理:
     $x^* = \operatorname{argmin} \|A \cdot x - p\|_2^2$  等价于伪逆  $x^* = (A^T A)^{-1} A^T \cdot p$ 
    rcond=None: 使用机器精度作为截断阈值, 处理近奇异矩阵 (欠定系统)
    """
    start_t = time.perf_counter()
    x_recon, _, _, _ = np.linalg.lstsq(A, p_vec, rcond=None)
    elapsed_ms = (time.perf_counter() - start_t) * 1000

    # clip 将可能出现的负值 (最小二乘的数学解不保证非负) 截断到 [0, 1];
    # _finite_clip 同时中和病态输入下的 NaN/Inf, 保证输出可显示。
    img_recon = _finite_clip(x_recon, n)
    return img_recon, elapsed_ms

# -----
# ART / SIRT 迭代重建
# -----

def compute_art(A: np.ndarray, p_vec: np.ndarray, n: int, n_iter: int,
               cancel_check: Callable[[], bool] | None = None,
               progress_cb: Callable[[int], None] | None = None) -> tuple[np.ndarray, float]:
    """ART (Kaczmarz 迭代) 重建, 返回 (img_recon, elapsed_ms)。

    A:      系统矩阵, shape=(n_rays, n2)
    p_vec:  弦图展平向量

```

```

n:          图像边长
n_iter:     迭代次数
cancel_check: 可选, cancel_check() 返回 True 时提前停止
progress_cb: 可选, progress_cb(it) 每迭代一轮调用

Kaczmarz 迭代公式 (逐射线更新):
"""
x ← x + (p_i - A_i·x) / ||A_i||² · A_i
"""
x = np.zeros(n * n, dtype=np.float32)
# einsum 'ij,ij->i': 逐行计算行向量的 L2 范数平方 ||A_i||²
# 预计算避免内层循环重复计算, 是 ART 的关键性能优化
row_norms_sq = np.einsum('ij,ij->i', A, A)

# 性能优化 (保 bit-exact): 循环外预筛"有效射线索引",
# 跳过 row_norms_sq[i] <= 1e-10 的全零行; 浮点除法保留原样, 结果与原版本逐 bit 一致。
valid_idx = np.flatnonzero(row_norms_sq > 1e-10)

start_t = time.perf_counter()
for it in range(n_iter):
    if cancel_check is not None and cancel_check():
        break
    for i in valid_idx:
        # 保留原 ART 更新公式: x += (p_i - A_i·x) / ||A_i||² · A_i
        # `+=` 已是 np.add(..., out=x) 原地累加, 仅 `scale * A[i]` 创建 1 个临时数组
        x += ((p_vec[i] - A[i] @ x) / row_norms_sq[i]) * A[i]
    np.clip(x, 0.0, None, out=x) # 非负约束, 原地避免新数组分配
    if progress_cb is not None:
        progress_cb(it)
elapsed_ms = (time.perf_counter() - start_t) * 1000

img_recon = _finite_clip(x, n)
return img_recon, elapsed_ms

def compute_sirt(A: np.ndarray, p_vec: np.ndarray, n: int, n_iter: int,
                 cancel_check: Callable[[], bool] | None = None,
                 progress_cb: Callable[[int], None] | None = None) -> tuple[np.ndarray, float]:
    """SIRT (同步迭代重建) 重建, 返回 (img_recon, elapsed_ms)。


SIRT 更新公式 (批量全射线更新):


$$x \leftarrow x + C \cdot A^T \cdot (R \cdot (p - A \cdot x))$$


其中  $C = \text{diag}(1/\text{列和})$ ,  $R = \text{diag}(1/\text{行和})$  是归一化矩阵 (用向量形式存储)。



相比 ART: 每次迭代计算量更大 (矩阵乘法), 但噪声鲁棒性更好, 收敛更平滑。


"""
x = np.zeros(n * n, dtype=np.float32)
col_sums = A.sum(axis=0) # 每列之和 = 每个像素被所有射线覆盖的总权重
row_sums = A.sum(axis=1) # 每行之和 = 每条射线穿过所有像素的总路径长度
# where 保护: 避免除以 0 (完全不被射线覆盖的像素列/行)
C = np.where(col_sums > 1e-10, 1.0 / col_sums, 0.0).astype(np.float32)

```

```

R = np.where(row_sums > 1e-10, 1.0 / row_sums, 0.0).astype(np.float32)

start_t = time.perf_counter()
for it in range(n_iter):
    if cancel_check is not None and cancel_check():
        break
    # x ← x + C·AT·(R·(p - A·x))
    # A @ x: 正向投影 (用当前估计值模拟弦图)
    # R * residual: 对每条射线按其路径长度归一化
    # A.T @ ...: 反投影 (将射线残差分配回各像素)
    # C * ...: 对每个像素按其被覆盖总权重归一化
    x = x + C * (A.T @ (R * (p_vec - A @ x)))
    x = np.clip(x, 0.0, None) # 非负约束
    if progress_cb is not None:
        progress_cb(it)
elapsed_ms = (time.perf_counter() - start_t) * 1000

img_recon = _finite_clip(x, n)
return img_recon, elapsed_ms
# ===== quantify.py (41 行) =====
# =====
# 器官定量计算模块
# 负责: 从分割蒙版 + HU 体积 + 体素尺寸算各器官的体积(mL)与平均 HU。
# 设计: 无任何 Qt/UI 依赖, 输入输出皆为普通 numpy 数组与 dict/list,
#       故可脱离 MedicalViewer 独立单元测试 (见 tests/test_gui.py::test_quantify)。
#       AnnotationMixin._compute_organ_stats 只是读取 self 状态后调用本函数的薄包装。
# =====

from __future__ import annotations

import numpy as np
import scipy.ndimage as ndimage

def compute_organ_stats(volume_hu: np.ndarray, volume_mask: np.ndarray,
                        spacing: tuple[float, float, float],
                        organ_names: dict[int, tuple[str, str]]) -> list[dict]:
    """统计 volume_mask 中各标签的体积(mL)与平均 HU, 按体积降序返回。

    volume_hu:    3D HU 值体素数组, shape=(Z,H,W)
    volume_mask:  同形状的 uint8 标签图 (0=背景, 1-255=器官/手动层)
    spacing:      (行间距 ps0, 列间距 ps1, 层厚 st), 单位 mm
    organ_names:  {标签号: (中文名, 英文名)}; 缺失标签回退为 "类{id}"/"cls{id}"
    返回:         [{ 'id', 'name_zh', 'name_en', 'voxels', 'volume_ml', 'mean_hu'}, ...],
                  按 volume_ml 降序; 无前景标签时返回 []。
    """
    ps0, ps1, st = spacing
    vox_ml = ps0 * ps1 * st / 1000.0 # 单体素体积, mm³ → mL
    counts = np.bincount(volume_mask.ravel(), minlength=256)
    present = [i for i in range(1, 256) if counts[i] > 0]

```



```

    if not present:
        return []
    # ndimage.mean 一次性算出所有标签区域的平均 HU, 避免逐类布尔索引
    means = np.atleast_1d(ndimage.mean(volume_hu, labels=volume_mask, index=present))
    rows = []
    for i, lid in enumerate(present):
        zh, en = organ_names.get(lid, (f"类{lid}", f"cls{lid}"))
        rows.append({'id': lid, 'name_zh': zh, 'name_en': en, 'voxels': int(counts[lid]),
                    'volume_ml': counts[lid] * vox_ml, 'mean_hu': float(means[i])})
    rows.sort(key=lambda r: -r['volume_ml'])
    return rows
# ===== segmentation.py (63 行) =====
#
# 肺分割数学降级纯计算模块
# 负责: 无 AI 模型 / ONNX 推理失败时, 用纯数学连通域算法从 HU 体积粗分割双肺。
# 设计: 无任何 Qt/UI/线程依赖, 输入 HU 数组, 输出 uint8 蒙版, 故可脱离
#       AutoAIEngineThread 独立单元测试 (见 tests/test_gui.py::test_lung_fallback)。
# =====

from __future__ import annotations

import numpy as np
import scipy.ndimage as ndimage

from constants import LUNG_FALLBACK_LABEL

def segment_lungs_fallback(volume_hu: np.ndarray, air_hu: float = -300.0,
                          label: int = LUNG_FALLBACK_LABEL) -> np.ndarray:
    """纯数学肺分割降级。返回 uint8 蒙版 (label=肺, 0=背景), 异常时返回全零。

    算法原理: 肺部在 CT 中为低密度空气区域 (HU < air_hu)。
    1. 阈值分割提取所有低密度区域 (空气 + 肺)
    2. 3D 连通域标记, 把相互接触的空气体素归为同一组
    3. 找出与六个边界面相交的连通域 -> 体外背景空气, 剔除
    4. 剩余内部空气再次连通域标记, 取体积最大者为主肺;
       若次大 ≥ 主肺体积 5% 则一并纳入 (双肺)
    """
    try:
        # 步骤1: 阈值分割, 提取所有低密度区域 (空气 + 肺部)
        air_mask = (volume_hu < air_hu).astype(np.uint8)

        # 步骤2: 3D 连通域标记, 把相互接触的空气体素归为同一组
        labels, _ = ndimage.label(air_mask)

        # 步骤3: 找出与六个边界面相交的连通域标签——这些是体外背景
        border_labels = (set(labels[0, :, :].flatten()) | set(labels[-1, :, :].flatten())
                        | set(labels[:, 0, :].flatten()) | set(labels[:, -1, :].flatten())
                        | set(labels[:, :, 0].flatten()) | set(labels[:, :, -1].flatten()))

```

```

# 步骤4: 从空气掩码中剔除所有边界连通域, 留下纯内部空气 (即肺)
internal_air = np.copy(air_mask)
for bl in border_labels:
    if bl != 0:
        internal_air[labels == bl] = 0

# 步骤5: 对内部空气再次连通域标记, 分离左右肺
labels_int, _ = ndimage.label(internal_air)
counts = np.bincount(labels_int.flatten())
counts[0] = 0 # 标签0是背景, 排除在外

# 步骤6: 取体积最大的连通域为主肺叶, 若第二大超过主肺的 5% 则一并纳入 (双肺)
mask = np.zeros_like(internal_air)
if len(counts) > 1:
    l1 = counts.argmax()
    mask[labels_int == l1] = label
    max_vol = counts[l1]
    counts[l1] = 0
    if counts.max() > max_vol * 0.05:
        l2 = counts.argmax()
        mask[labels_int == l2] = label
    return mask
except Exception:
    # 任何异常均返回全零掩码, 保证调用方 (UI) 不崩溃
    return np.zeros_like(volume_hu, dtype=np.uint8)
# ===== mpr_geometry.py (51 行) =====
# MPR 坐标几何纯计算模块
# 负责: MPR 三平面与 3D 光标 [z,y,x] 之间的坐标换算, 以及双序列解剖 z 配准。
# 设计: 无任何 Qt/UI 依赖, 纯整数/数组运算。把原先散落在 sync_crosshair /
# _render_clinical_plane / _render_compare 里的同一套坐标约定收拢为单一
# 可信来源 (历史上轴向易错, 见 BUG J), 并可独立单测。
#
# 坐标约定 (三平面共用同一个 3D 光标 [z, y, x]):
# - Axial 视图 (px,py) → 3D 的 (x=px, y=py), z 不变
# - Coronal 视图 (px,py) → 3D 的 (x=px, z=py), y 不变
# - Sagittal 视图 (px,py) → 3D 的 (y=px, z=py), x 不变
# =====

from __future__ import annotations

import numpy as np

from constants import AXIAL, CORONAL, SAGITTAL

def hover_to_voxel(plane: int, px: int, py: int,
                  cur: tuple[int, int, int],
                  shape: tuple[int, int, int]) -> tuple[int, int, int]:
    """把某平面上的悬停像素 (px,py) 映射为完整 3D 体素 (z,y,x), 非该平面的轴沿用当前

```

```

    光标 cur=(z,y,x), 并按 shape=(Z,Y,X) 裁剪到体积范围内。返回 (z,y,x)。”””
    z, y, x = cur
    Z, Y, X = shape
    if plane == AXIAL:
        x, y = px, py
    elif plane == CORONAL:
        x, z = px, py
    elif plane == SAGITTAL:
        y, z = px, py
    x = max(0, min(x, X - 1))
    y = max(0, min(y, Y - 1))
    z = max(0, min(z, Z - 1))
    return z, y, x

def voxel_to_crosshair(plane: int, z: int, y: int, x: int) -> tuple[int, int]:
    """把 3D 光标 (z,y,x) 投影为某平面上十字准线的 2D 坐标 (cx,cy)。”””
    if plane == CORONAL:
        return x, z
    if plane == SAGITTAL:
        return y, z
    return x, y # AXIAL (含缺省)

def nearest_slice(zpos, target_z: float) -> int:
    """在解剖 z 坐标数组 zpos 中, 返回与 target_z 最接近的切片索引 (双序列配准)。”””
    return int(np.argmax(np.abs(np.asarray(zpos) - target_z)))
# ===== constants.py (67 行) =====
# =====
# 全局常量定义模块
# 负责: 跨 UI 与业务逻辑层共享的工具/平面/视图标识符
# 提取动机: 原先散落在 graphics_view.py (UI 组件模块) 中却被 main.py 业务层
#           大量引用, 导致业务层不必要地依赖 UI 组件模块; 独立 constants.py
#           让两侧都从中性模块导入, 解开耦合。
# =====

import os as _os

import numpy as _np

# 工具栏工具 ID 枚举 (用整数而非 Enum 方便与 QPushButton.clicked 直接对接)
# 前 6 个为原有工具; 6/7=分割修正 (画笔/橡皮); 8=椭圆 ROI 密度测量
(TOOL_POINTER, TOOL_RULER, TOOL_DRAW, TOOL_CROP, TOOL_RECT_CROP, TOOL_AI_TRACK,
 TOOL_SEG_BRUSH, TOOL_SEG_ERASE, TOOL_ROI) = range(9)

# MPR 三平面常量, 与 combo_plane 下拉框的索引严格对应
AXIAL = 0 # 横断面: 沿 Z 轴切片, 最常用的阅片视角
CORONAL = 1 # 冠状面: 沿 Y 轴切片, 前后方向
SAGITTAL = 2 # 矢状面: 沿 X 轴切片, 左右方向

```

```
# =====
# AI 多器官分割相关常量
# =====
# organs.onnx: 25 类胸腹多器官分割模型 (含 5 个肺叶类)。
# 外部权重 organs.onnx.data 必须与 .onnx 同目录 (ONNX 相对 onnx 文件路径解析)。
MODEL_PATH = _os.path.join(_os.path.dirname(_os.path.abspath(__file__)), "models", "organs.onnx")
# 器官名候选表 (由真实数据解剖推断, 可被官方标签表替换)
LABELS_JSON = _os.path.join(_os.path.dirname(_os.path.abspath(__file__)), "models", "organ_labels_candidate.json")

# 手动 3D 追踪结果占用的专属标签值, 避开 0~24 的模型类别, 防止与器官语义冲突
MANUAL_TRACK_LABEL = 255
# 数学降级算法 (无模型时) 分出的肺, 复用右肺叶标签以显示为肺色
LUNG_FALLBACK_LABEL = 10

# 25 类器官调色板 (RGB)。索引即类别号; 0=背景在 LUT 中设为全透明。
# 肺叶用红橙 (右)/蓝青 (左) 区分左右; 心脏粉、气管黄、腹部器官各异。
LABEL_COLORS = {
    1: (229, 57, 53),      # 主动脉/大血管
    5: (236, 64, 122),     # 心脏
    6: (161, 136, 127),    # 肠道/直肠气体
    10: (239, 83, 80),     # 右肺叶
    11: (255, 138, 101),   # 右肺叶
    12: (66, 165, 245),    # 左肺叶
    13: (41, 182, 246),    # 左肺叶
    14: (38, 198, 218),    # 左肺叶
    15: (120, 144, 156),   # 颈部软组织
    16: (255, 235, 59),    # 气管
    18: (156, 204, 101),   # 脾
    19: (255, 213, 79),    # 膀胱?
    20: (255, 167, 38),    # 胃
    21: (141, 110, 99),    # 肝
    22: (224, 224, 224),   # 骨/结肠?
    23: (126, 87, 194),    # 肾/肾上腺?
    MANUAL_TRACK_LABEL: (255, 255, 255), # 手动 3D 追踪: 白色
}
# 未列出的 unknown 类 (2,3,4,7,8,9,17,24) 落到此暗灰, 基本不出现
_UNKNOWN_COLOR = (96, 96, 96)
_MASK_ALPHA = 110 # 蒙版叠加透明度 (0~255)

# 预构建 (256,4) RGBA 查找表: ov = LABEL_LUT[slice_labels] 一步向量化上色
LABEL_LUT = _np.zeros((256, 4), dtype=_np.uint8)
for _i in range(1, 25):
    _c = LABEL_COLORS.get(_i, _UNKNOWN_COLOR)
    LABEL_LUT[_i] = (*_c, _MASK_ALPHA)
LABEL_LUT[MANUAL_TRACK_LABEL] = (*LABEL_COLORS[MANUAL_TRACK_LABEL], _MASK_ALPHA)
```